

الجمهورية العربية السورية

المعهد العالي للعلوم التطبيقية والتكنولوجيا

قسم المعلوماتية .

العام الدراسي 2025/2026 .

تصميم و تنفيذ شبكة SD-WAN ذاتية الادارة و التعافي باستخدام وكيل ذكاء صناعي و تقنيات الشبكات المعرفة برمجيا .

إعداد

شعيب صبح .

إشراف

م.محمد بشار دسوقي

د.حسين خيو

.

إهداء

أهدي هذا العمل إلى والديّ الكريمين، تقديراً لما قدّماه لي من دعمٍ وتشجيعٍ وصبرٍ، ولما كان لهما من فضلٍ كبيرٍ في مسيرتي العلمية.

وإلى أساتذتي الذين كان لعلمهم وتوجيههم وإرشادهم أثرٌ أساسي في تكويني الأكاديمي، وفي الوصول إلى هذه المرحلة.

وإلى عائلتي وأصدقائي وكل من ساندني ووقف إلى جانبي خلال سنوات الدراسة، وشارك معي لحظات التعب والنجاح.

وإلى كل من يسعى إلى العلم والمعرفة، ويؤمن بأن التطور يبدأ بفكرة، وأن الإصرار والعمل قادران على تحويلها إلى إنجاز.

وإلى كل من كان له أثر، ولو بكلمة، في إنجاز هذا العمل، أهدي هذا الجهد المتواضع.

كلمة شكر...

أتقدم بجزيل الشكر وعظيم الامتنان إلى أساتذتي في المعهد العالي للعلوم التطبيقية والتكنولوجيا، لما قدموه لنا من علمٍ ومعرفةٍ وتوجيهٍ خلال سنوات الدراسة، وما بذلوه من جهدٍ في سبيل إعدادنا العلمي والمهني.

وأخص بالشكر الدكتور حسين خيو والأستاذ محمد بشار دسوقي، على إشرافهما وتوجيهاتهما وملاحظاتهما القيمة التي أسهمت في تطوير هذا المشروع وإخراجه بالصورة الحالية، وعلى ما قدماه من دعمٍ ومتابعةٍ خلال مراحل إنجاز العمل.

كما أتوجه بالشكر إلى جميع أعضاء الهيئة التدريسية في قسم المعلوماتية، الذين كان لجهودهم أثرٌ مهم في بناء المعارف والمهارات التي اعتمدت عليها في تنفيذ هذا المشروع.

وفي الختام، أتوجه بالشكر لكل من أسهم، بشكل مباشر أو غير مباشر، في إنجاز هذا العمل، سائلاً الله أن يكون هذا المشروع خطوةً جديدةً في مسيرتي العلمية والعملية، وأن يكون نافعاً ومفيداً.

خلاصة

أصبحت الشبكات الحديثة أكثر تعقيداً نتيجة ازدياد عدد التطبيقات والخدمات التي تعتمد عليها، وارتفاع متطلبات الأداء والموثوقية، والحاجة إلى الاستجابة السريعة للأعطال والتغيرات في حالة الشبكة. ومن هنا تظهر أهمية تقنيات Software-Defined Networking (SDN) وSoftware-Defined Wide Area Networking (SD-WAN) التي تسمح بفصل منطق التحكم عن عملية تمرير البيانات وإدارة الشبكة بصورة مركزية وأكثر مرونة.

يقدم هذا المشروع نظاماً بعنوان Self-Healing and Self-Managed SD-WAN يهدف إلى تطوير شبكة قادرة على مراقبة حالتها التشغيلية، واكتشاف التغيرات أو الأعطال التي تؤثر في جودة الاتصال، وتحليل حالة المسارات المتاحة، ثم اتخاذ قرار مناسب لإعادة توجيه حركة المرور عند الحاجة.

يعتمد النظام على بنية متعددة المكونات تشمل طبقة لمراقبة الشبكة وجمع القياسات، وطبقة لتحليل السياسات واتخاذ القرار، وطبقة لتنفيذ القرارات من خلال وحدة التحكم بالشبكة. ويتم جمع مجموعة من مؤشرات الأداء، مثل استخدام النطاق الترددي، وزمن التأخير، والتذبذب (Jitter)، وفقدان الحزم، واستخدامها لتقييم جودة الروابط والمسارات.

تم استخدام Ryu كوحدة تحكم شبكية تعتمد على OpenFlow، و Open vSwitch (OVS) لتنفيذ عمليات تمرير الحزم، و ContainerNet لبناء بيئة الاختبار، إضافة إلى استخدام VXLAN لبناء شبكة Overlay بين عقد الشبكة. كما تم استخدام آليات القياس والمراقبة لتقييم أداء الأنفاق والمسارات، وتخزين البيانات اللازمة للتحليل، مع إمكانية عرض مؤشرات الأداء بصورة آنية.

يركز المشروع كذلك على آلية اختيار المسار اعتماداً على متجه من مؤشرات جودة الشبكة، بحيث لا يعتمد القرار على أقصر مسار من حيث عدد القفزات، وإنما يأخذ في الاعتبار حالة الشبكة الفعلية. وعند اكتشاف تدهور في المسار الحالي، يمكن للنظام اختيار مسار بديل مناسب وتنفيذ عملية إعادة التوجيه، بما يحقق مفهوم Self-Healing ويقلل من تأثير الأعطال والتدهور على حركة البيانات.

تم اختبار النظام ضمن بيئة افتراضية من خلال إنشاء سيناريوهات مختلفة لحالة الشبكة، بما في ذلك التأخير وفقدان الحزم وتغير استخدام الروابط، ثم مراقبة استجابة النظام لهذه التغيرات. وتوفر النتائج أساساً لتقييم قدرة النظام على اكتشاف التدهور والاستجابة له بشكل آلي، مع إمكانية تطويره مستقبلاً ليشمل سياسات أكثر تقدماً لاتخاذ القرار واستخدام تقنيات الذكاء الاصطناعي والتعلم الآلي بصورة أوسع.

Abstract

Modern computer networks have become increasingly complex due to the growing number of applications and services that rely on them, as well as the increasing requirements for performance, availability, and reliability. Software-Defined Networking (SDN) and Software-Defined Wide Area Networking (SD-WAN) provide flexible approaches for managing such networks by separating the control logic from the data forwarding plane and enabling centralized and programmable network control.

This project, entitled “Self-Healing and Self-Managed SD-WAN”, proposes a network architecture capable of monitoring its operational state, detecting failures and performance degradation, evaluating available paths, and automatically responding to network conditions by rerouting traffic when necessary.

The proposed system consists of several cooperating components, including a telemetry layer responsible for collecting network measurements, a policy and decision layer responsible for analyzing the collected information and selecting appropriate actions, and an execution layer responsible for applying the resulting decisions to the network through the SDN controller.

Several network performance metrics are considered, including bandwidth utilization, latency, jitter, and packet loss. These measurements are used to evaluate the quality of links and paths instead of relying solely on traditional shortest-path metrics. This allows the system to make routing decisions based on the actual network conditions.

The implementation uses Ryu as an OpenFlow-based SDN controller and Open vSwitch (OVS) as the programmable forwarding infrastructure. Containernet is used to construct the experimental network environment, while VXLAN is used to establish an overlay network between selected edge switches. Monitoring and telemetry components are used to collect and store network measurements and evaluate tunnel and path performance.

The proposed system also incorporates a path-scoring mechanism based on a vector of network-performance features. When the current path becomes degraded, the system can evaluate alternative paths and select a more suitable one according to the observed network conditions. The selected decision can then be enforced through the SDN control plane, providing an automated response to network failures and performance degradation.

The system is evaluated in a controlled virtual network environment through different network impairment scenarios, including increased latency, packet loss, and changes in link utilization. The collected measurements are used to assess the monitoring and rerouting mechanisms and to evaluate the system's ability to respond automatically to changing network conditions.

The project demonstrates the feasibility of combining SDN programmability, network telemetry, VXLAN-based overlays, and automated decision-making to build a more adaptive and resilient SD-WAN environment. Future work may extend the decision-

making mechanism through more advanced artificial intelligence and machine-learning techniques.

فهرس المحتويات

II	 خلاصة
V	 فهرس المحتويات
VI	 قائمة الأشكال
VI	 قائمة الجداول
1	 مقدمة
2	 الفصل الأول
4	 1.1- المتطلبات الوظيفية
5	 2.1- المتطلبات غير الوظيفية
7	 الفصل الثاني
7	 1.2- مخططات حالات الاستخدام
8	 2.2- الوصف النصي لحالات الاستخدام :
12	 3.2- مخطط المفاهيم (مخطط الصفوف الأولى)
13	 الفصل الثالث
13	 1.3- التصميم الأولي للنظام
15	 2.3- التصميم التفصيلي
15	 1.2.3- مخطط الصفوف التصميمي
16	 2.2.3- تصميم قاعدة المعطيات
17	 3.2.3- تصميم البنيان الفيزيائي
17	 4.2.3- تصميم الخوارزميات (الإجرائيات Processes)
23	 5.2.3- تصميم الواجهات
25	 الفصل الرابع
25	 1.4- بيئة العمل وأدوات التنفيذ
26	 2.4- الاختبارات والأمثلة :
36	 3.4- خلاصة وآفاق مستقبلية

قائمة الأشكال

7	الشكل 1: مخطط حالات الاستخدام
12	الشكل 2: مخطط المفاهيم
14	الشكل 3: الطبقات المكونة للنظام
15	الشكل 4: مخطط الصفوف التفصيلي
16	الشكل 5: تصميم قاعدة المعطيات
17	الشكل 6: تصميم البنية الفيزيائي
24	الشكل 7: مخطط توزيع الشاشة الخاص بواجهة المراقبة و التحكم
27	الشكل 8: الطوبولوجيا 1
31	الشكل 9: الطوبولوجيا 2
33	الشكل 10: لمحة عامة عن الشبكة
33	الشكل 11: overlay topology
33	الشكل 12: underlay topology
34	الشكل 13: مؤشرات أداء الأنفاق
34	الشكل 14: مؤشرات أداء الوصلات الفيزيائية
35	الشكل 15: لوحة التحكم الخاصة بمحاكاة حقن الأعطال
35	الشكل 16: سجل عمليات المحاكاة
35	الشكل 17: سجل القرارات المتخذة
36	الشكل 18: سجل أحداث الشبكة

قائمة الجداول

9	الجدول 1: الوصف النصي لحالة الاستخدام inject link fault
11	الجدول 2: سيناريو النجاح الخاص بحالة inject link fault
15	الجدول 3: الطبقات المكونة للنظام و وظائفها
25	الجدول 4: بيئة التشغيل الخاصة بالنظام

مقدمة

شهد الإنترنت وشبكات الحاسوب تطورًا متسارعًا خلال العقود الأخيرة، وأصبحت بنية تحتية أساسية تقوم عليها معظم مناحي الحياة الرقمية، مما فرض تحديات متزايدة في ضمان أدائها وموثوقيتها. ومع اتساع الشبكات وتعقيدها، لم تعد أساليب الإدارة التقليدية القائمة على الإعداد اليدوي والتدخل البشري كافية للاستجابة السريعة للأعطال، خصوصًا عندما تتمثل المشكلة بتدهور تدريجي في جودة الخدمة وليس انقطاعًا كاملاً للاتصال.

ظهرت تقنية الشبكات المعرفة بالبرمجيات (SDN) كحل لهذه المشكلة، من خلال فصل منطق التحكم عن مستوى تمرير البيانات ووضعه في وحدة تحكم مركزية قابلة للبرمجة. وقد فتح ذلك المجال أمام بناء شبكات قادرة على مراقبة نفسها وتحليل حالتها واتخاذ قرارات تصحيحية آلية — وهو ما يُعرف بمفهوم الشبكة ذاتية الإصلاح (Self-Healing Network). إلا أن ذلك يتطلب طبقة متكاملة لجمع القياسات وتحليلها واتخاذ قرار مناسب والتحقق من صحته قبل تنفيذه، دون الاعتماد الكامل على التدخل البشري.

يهدف هذا المشروع إلى تصميم وتنفيذ نظام Self-Healing and Self-Managed SD-WAN قادر على اكتشاف طوبولوجيا الشبكة تلقائيًا، وجمع مؤشرات الأداء (التأخير، التذبذب، فقدان الحزم، النطاق الترددي)، وبناء أنفاق VXLAN لقياس جودة الاتصال. End-to-End كما يستخدم نموذج تعلم معزز لاختيار مسار بديل عند اكتشاف تدهور، مع طبقة تحقق أمان (Guardrail) قبل تنفيذ أي قرار عبر وحدة التحكم Ryu، ولوحة مراقبة رسومية لمتابعة حالة الشبكة وقرارات النظام.

قُسِّم هذا التقرير إلى أربعة فصول: يتناول **الفصل الأول** التعريف بالمشروع ومتطلباته. ويستعرض **الفصل الثاني** تحليل النظام عبر مخططات حالات الاستخدام والمخطط المفاهيمي. ويتناول **الفصل الثالث** التصميم التفصيلي (المعماري، قاعدة البيانات، الخوارزميات، الواجهات). وأخيرًا يعرض **الفصل الرابع** بيئة التنفيذ وخطة الاختبارات والخلاصة والآفاق المستقبلية.

الفصل الأول

التعريف بالمشروع

نقدم في هذه الفصل تمهيداً لموضوع البحث وأهم الحوافر التي أدت لإختياره ثم يتم عرض النتائج والإسهامات وفي نهاية هذا الفصل نعرض هيكلية الأطروحة وطريقة تنظيمها.

شهدت شبكات الحاسوب خلال السنوات الأخيرة تطوراً كبيراً نتيجة الانتشار الواسع للخدمات الرقمية والتطبيقات الموزعة والحوسبة السحابية، الأمر الذي أدى إلى زيادة الاعتماد على الشبكات في نقل البيانات وتوفير الخدمات المختلفة. ومع ازدياد حجم الشبكات وتعقيدها، أصبحت المحافظة على الأداء والاستمرارية والموثوقية من أهم التحديات التي تواجه مسؤولي الشبكات.

تعتمد آليات إدارة الشبكات التقليدية في كثير من الحالات على إعداد أجهزة الشبكة بصورة يدوية، كما أن الاستجابة للأعطال أو التغيرات في حالة الشبكة قد تتطلب تدخلاً بشرياً. ويؤدي ذلك إلى زيادة زمن الاستجابة للأعطال، وخاصة عندما تكون المشكلة مرتبطة بتدهور تدريجي في جودة أحد الروابط أو المسارات وليس بانقطاع كامل للاتصال.

ظهرت تقنية Software-Defined Networking (SDN) كأحد الأساليب التي تهدف إلى جعل إدارة الشبكات أكثر مرونة وقابلية للبرمجة، وذلك من خلال فصل مستوى التحكم عن مستوى تمرير البيانات. ويسمح هذا الفصل بوضع منطق التحكم واتخاذ القرار في وحدة تحكم برمجية يمكنها الحصول على معلومات عن حالة الشبكة وتطبيق سياسات التوجيه بصورة مركزية.

وبالاستفادة من هذه المبادئ ظهرت تقنية Software-Defined Wide Area Networking (SD-WAN)، التي تهدف إلى تحسين إدارة شبكات WAN من خلال استخدام التحكم البرمجي والسياسات الديناميكية والاستفادة من عدة مسارات أو تقنيات اتصال. ومن أهم المزايا التي يمكن تحقيقها في هذا السياق القدرة على اختيار المسار المناسب اعتماداً على حالة الشبكة ومتطلبات التطبيقات.

إلا أن وجود وحدة تحكم مركزية وإمكانية برمجة الشبكة لا يعني بالضرورة أن الشبكة قادرة على إصلاح نفسها تلقائياً. إذ تحتاج عملية Self-Healing إلى آلية مستمرة لمراقبة الشبكة، وجمع القياسات المتعلقة بأداء الروابط والمسارات، وتحليل هذه القياسات، واكتشاف حالات التدهور، ثم اتخاذ وتنفيذ إجراء مناسب دون الاعتماد الكامل على التدخل البشري.

من هنا تأتي فكرة هذا المشروع، الذي يهدف إلى بناء نظام Self-Healing and Self-Managed SD-WAN قادر على مراقبة الشبكة وتحليل حالتها واتخاذ قرارات آلية لتحسين استمرارية الاتصال. ويعتمد النظام على جمع مجموعة من مؤشرات الأداء، من أهمها زمن التأخير، والتذبذب، وفقدان الحزم، واستخدام النطاق الترددي، ثم استخدام هذه المؤشرات في تقييم المسارات المتاحة واختيار المسار الأكثر ملاءمة للحالة الحالية للشبكة.

تم بناء بيئة تجريبية تعتمد على Containernet لإنشاء عناصر الشبكة الافتراضية، و Open vSwitch لتنفيذ عملية تمرير البيانات، و Ryu لتنفيذ وظائف التحكم البرمجية باستخدام OpenFlow. كما تم استخدام VXLAN لبناء شبكة Overlay تربط عقد الشبكة الطرفية، بما يسمح بدراسة سلوك حركة البيانات والقياسات ضمن بيئة قريبة من مفهوم شبكات SD-WAN الحديثة.

ويولي المشروع أهمية خاصة لطبقة Telemetry، إذ لا يمكن اتخاذ قرار توجيه مناسب دون امتلاك معلومات دقيقة عن حالة الشبكة. لذلك يتم جمع قياسات مختلفة من الشبكة واستخدامها في بناء صورة عن حالة الروابط والمسارات. وتشمل هذه القياسات مؤشرات مرتبطة بالأداء وجودة الخدمة SLA، مثل latency و jitter و bandwidth utilization. و packet loss.

بعد جمع القياسات، يتم تحليلها واستخدامها في تقييم المسارات. وبدل الاعتماد فقط على عدد القفزات أو أقصر مسار من الناحية الطوبولوجية، يأخذ النظام في الاعتبار مؤشرات الأداء الفعلية للروابط التي يتكون منها المسار. وبذلك يمكن أن يكون المسار الأطول من حيث عدد القفزات هو الخيار الأفضل إذا كانت روابطه توفر جودة اتصال أعلى من المسار الأقصر.

وعند اكتشاف تدهور في المسار الحالي، يقوم النظام بتقييم المسارات البديلة المتاحة، والتحقق من صلاحية القرار المقترح وفق مجموعة من القواعد والسياسات، ثم إرسال القرار إلى طبقة التنفيذ لتطبيق قواعد إعادة التوجيه المناسبة في الشبكة. وتسمح هذه الآلية بتحقيق استجابة آلية لحالات الأعطال أو التدهور، وهو ما يشكل الأساس لمفهوم الشبكة ذاتية الإصلاح.

يتضمن المشروع كذلك آليات لقياس أداء الأنفاق المبنية باستخدام VXLAN، وذلك من خلال تنفيذ اختبارات تعتمد على Ping و iperf3 وجمع مؤشرات التأخير والتذبذب وفقدان الحزم وعرض النطاق الترددي. وتستخدم هذه القياسات في تقييم جودة الاتصال بين نقاط الشبكة الطرفية وفي دراسة تأثير تغيرات الشبكة على المسارات.

1.1- المتطلبات الوظيفية

يجب على النظام تنفيذ لائحة المتطلبات التالية:

1. اكتشاف ومراقبة طوبولوجيا الشبكة : يجب أن يكون النظام قادراً على اكتشاف طوبولوجيا شبكة-SD WAN بشكل آلي وتحديثها بصورة مستمرة، بحيث يتم التعرف على المحولات والروابط والمنافذ التي تربط بينها والمسارات الممكنة بين المحولات. بالإضافة لجمع بيانات الأداء بشكل دوري، بما في ذلك استخدام عرض النطاق (Utilization) ، وزمن الوصول (Latency) ، والتذبذب (Jitter) ، وفقد الحزم (Packet Loss) ، بالإضافة إلى مؤشرات SLA الخاصة بأنفاق VXLAN .
2. إنشاء وإدارة أنفاق VXLAN : يجب أن يكون النظام قادراً على إنشاء وإدارة أنفاق VXLAN بين محولات الحافة، بما في ذلك إدارة نقاط النهاية VTEP و VNI ومعلومات الأنفاق، بحيث توفر الأنفاق اتصالاً بين مواقع الشبكة وتسمح بقياس جودة الاتصال بصورة End-to-End .
3. اكتشاف التدهور وتوليد المسارات البديلة : يجب أن يكتشف النظام حالات تدهور الشبكة مثل WARNING و CRITICAL و UNREACHABLE وعند الحاجة يقوم بتوليد مجموعة من المسارات البديلة بين طرفي النفق المتأثر، مع استبعاد المسار النشط من خيارات إعادة التوجيه.
4. اتخاذ القرار باستخدام Reinforcement Learning : يجب أن يستخدم النظام نموذج Reinforcement Learning قائم على Linear Function Approximation لتقييم المسارات المرشحة اعتماداً على متجه يتكون من Utilization و Latency و Jitter و Packet Loss ويتم اختيار الإجراء باستخدام سياسة ϵ -greedy التي توازن بين Exploration و Exploitation، مع اعتبار البقاء على المسار الحالي (NONE) أحد الخيارات الممكنة.
5. لتحقيق الأمن من قرارات إعادة التوجيه : يجب أن تمر قرارات REROUTE عبر طبقة Guardrail قبل تنفيذها للتحقق من صحة المسار المختار والتأكد من وجوده ضمن المسارات المرشحة وأنه يختلف عن المسار النشط الحالي. وفي حال عدم استيفاء شروط التحقق، يتم رفض القرار ومنع تنفيذه .
6. تنفيذ إعادة التوجيه : عند الموافقة على القرار، يجب أن يكون النظام قادراً على تنفيذ عملية REROUTE باستخدام Ryu Controller و OpenFlow من خلال تثبيت قواعد تحويل ذات أولوية أعلى على المحولات، مع دعم إعادة توجيه حركة النفق في الاتجاهين بين طرفي الاتصال.
7. تقييم نتيجة القرار والتعلم Online : بعد تنفيذ عملية إعادة التوجيه، يجب أن يعيد النظام قياس SLA للنفق بعد فترة استقرار، ثم يحسب Reward بناءً على التغير في Latency و Jitter و Packet Loss قبل وبعد

العملية. وتستخدم هذه المكافأة لتحديث أوزان نموذج RL باستخدام Online SGD، بحيث يتعلم النظام من نتائج قراراته الفعلية.

8. تسجيل وحفظ عمليات النظام : جب أن يقوم النظام بتخزين بيانات القياس وقرارات إعادة التوجيه ونتائج تنفيذها وقيم Reward وحالات SLA في قاعدة بيانات، كما يجب أن يحفظ أوزان نموذج RL بحيث يمكن استعادتها بعد إعادة تشغيل النظام وإعادة استخدام الخبرات السابقة .
9. يجب أن يوفر النظام آلية لتدريب نموذج PathScorer قبل التشغيل الكامل باستخدام بيانات اصطناعية (Synthetic Warm-up)، بالإضافة إلى إمكانية إعادة بناء النموذج باستخدام الانتقالات الحقيقية المخزنة في قاعدة البيانات (Database Replay) ، بهدف تحسين أداء النموذج وتقليل فترة التعلم الأول .
10. يجب أن يوفر النظام لوحة مراقبة رسومية Dashboard: تتيح للمستخدم متابعة حالة الشبكة والروابط وأنفاق VXLAN ومؤشرات SLA ، بالإضافة إلى عرض المسارات الحالية والبديلة وقرارات RL ونتائج ال Guardrail والتنفيذ وقيم Reward والتغير في الأداء قبل وبعد عمليات إعادة التوجيه

2.1- المتطلبات غير الوظيفية

1. الأداء (Performance) : يجب أن يعمل النظام بكفاءة في جمع بيانات الشبكة وتحليلها واتخاذ قرارات إعادة التوجيه دون التسبب في حمل كبير على مكونات الشبكة أو وحدة التحكم. كما يجب أن تكون عمليات حساب المسارات وتقييمها باستخدام نموذج RL سريعة بما يكفي للاستجابة لحالات التدهور ضمن فترة زمنية مناسبة .
2. الاستجابة في الزمن الحقيقي (Real-Time Responsiveness) : يجب أن يكون النظام قادراً على اكتشاف التغيرات المهمة في حالة الشبكة والاستجابة لها خلال فترة زمنية قصيرة، بحيث يتم جمع القياسات بصورة دورية وتحليلها واتخاذ قرار إعادة التوجيه عند الحاجة دون تأخير يؤثر بشكل كبير على أداء الخدمة.
3. الاعتمادية (Reliability) : يجب أن يحافظ النظام على عمل مكوناته الأساسية بصورة مستقرة حتى عند حدوث تغيرات أو أعطال في الشبكة. كما يجب ألا يؤدي فشل عملية قياس أو فقدان بعض البيانات إلى توقف النظام بالكامل، بل يتم التعامل مع البيانات غير المتوفرة بطريقة آمنة والاستمرار في تشغيل بقية المكونات .
4. التعافي الذاتي (Self-Healing Capability) : يجب أن يتمتع النظام بالقدرة على اكتشاف حالات تدهور الخدمة واتخاذ إجراءات تصحيحية بصورة آلية دون الحاجة إلى تدخل بشري مباشر في كل عملية. ويجب أن يعتمد نجاح عملية التعافي على القياسات الفعلية بعد التنفيذ، بحيث يمكن للنظام تقييم قراره وتحسين قراراته المستقبلية .

5. السلامة والتحكم بالقرارات (Safety) : يجب ألا يسمح النظام بتنفيذ قرارات الذكاء الاصطناعي بصورة مباشرة دون التحقق منها. ولذلك يجب أن تمر عمليات إعادة التوجيه عبر طبقة Guardrail للتحقق من صحة المسار والهدف قبل تنفيذها، مما يقلل من احتمالية قيام نموذج التعلم باتخاذ قرار يؤدي إلى مسار غير صحيح أو إعادة توجيه غير ضرورية .
6. قابلية التوسع (Scalability) : يجب أن يكون تصميم النظام قابلاً للتوسع لإضافة محولات وروابط وأنفاق ومسارات جديدة دون الحاجة إلى إعادة تصميم المكونات الأساسية. كما يجب أن يسمح التصميم بإضافة نماذج أو خوارزميات تعلم أكثر تطوراً مستقبلاً دون تغيير الواجهات الرئيسية بين مكونات النظام.
7. قابلية الصيانة والتطوير (Maintainability) : يجب أن يكون النظام مقسماً إلى مكونات مستقلة نسبياً مثل Telemetry وSLA Monitoring وRL Decision وGuardrail وExecution وDashboard، بحيث يمكن تعديل أو استبدال أحد المكونات دون التأثير بشكل كبير على بقية النظام. كما يجب أن يكون نموذج PathScorer قابلاً للاستبدال مستقبلاً بنموذج أكثر تعقيداً دون تغيير آلية استدعائه الأساسية .
8. قابلية المراقبة والتفسير (Observability and Explainability) : يجب أن يوفر النظام معلومات كافية لمراقبة وفهم قراراته، بما في ذلك المسارات المرشحة والمسار المختار ونوع القرار، وما إذا كان القرار ناتجاً عن Exploration أو Exploitation ، بالإضافة إلى نتيجة الـ Guardrail والتنفيذ وقيمة Reward ويساعد ذلك المستخدم على تحليل سلوك النظام والتحقق من صحة قرارات الذكاء الاصطناعي .
9. قابلية الاستعادة (Recoverability) : يجب أن يكون النظام قادراً على استعادة حالته الأساسية بعد إعادة تشغيل مكوناته، وذلك من خلال حفظ بيانات القياس وقرارات النظام وسجل عمليات RL وأوزان النموذج. ويسمح ذلك باستمرار التعلم والاستفادة من الخبرات السابقة بدلاً من بدء عملية التعلم من الصفر بعد كل عملية إعادة تشغيل .
10. دقة القياس (Measurement Accuracy) : يجب أن تعتمد قرارات النظام على بيانات قياس تمثل الحالة الفعلية للشبكة قدر الإمكان. لذلك يجب أن يتم جمع مؤشرات الروابط والـ SLA بصورة دورية، مع التمييز بين القيمة الصفرية والقيمة غير المقاسة وعدم اعتبار غياب القياس دليلاً على أن حالة الرابط أو النفق سليمة .
11. سهولة الاستخدام (Usability) : يجب أن توفر واجهة الـ Dashboard عرضاً واضحاً ومفهوماً لحالة الشبكة ونتائج القياس وقرارات النظام، بحيث يستطيع المستخدم متابعة أداء الشبكة وفهم عمليات إعادة التوجيه ونتائجها دون الحاجة إلى تحليل سجلات النظام أو تنفيذ أوامر يدوية بشكل مستمر.

الفصل الثاني

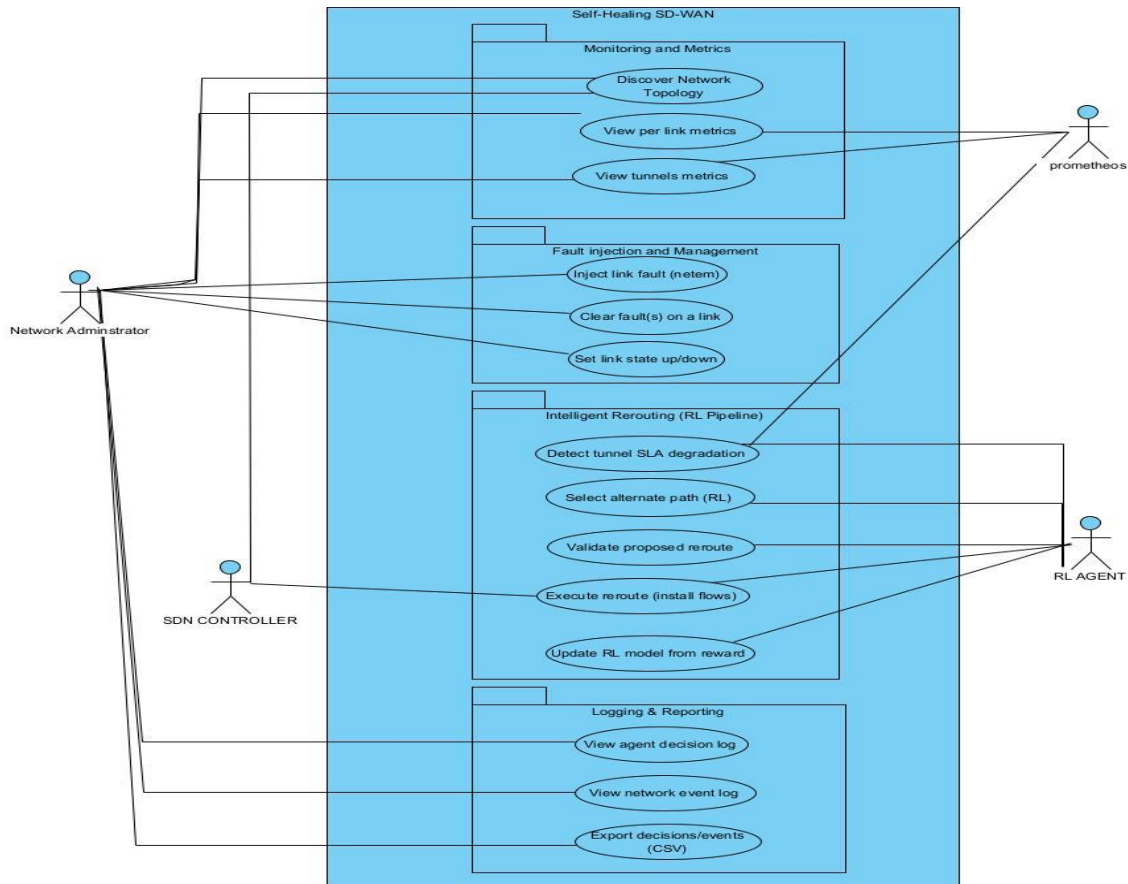
تحليل المتطلبات

SRA (Software Requirements Analysis)

نقدم في هذه الفصل تحليلاً مفصلاً لما ذكرناه في الفصل الأول من متطلبات (وظيفية وغير وظيفية)، حيث نورد مخططات حالات الاستخدام مع الوصف النصي الخاص بأحد الحالات، ونورد مخططاً أولياً للصفوف (مخطط المفاهيم).

1.2- مخططات حالات الاستخدام

نستعرض في هذا القسم مخطط حالات الاستخدام مقسماً إلى عدة أقسام وذلك بغرض التوضيح فقط.



الشكل 1: مخطط حالات الاستخدام

2.2- الوصف النصي لحالات الاستخدام :

نقدم في هذا القسم الوصف النصي لأحد حالات الاستخدام المبينة في الشكل أعلاه.

اسم الحالة: حقن عطل في وصلة الشبكة (Inject Link Fault)	
الوصف Description	تتيح هذه الحالة لمدير الشبكة محاكاة عطل أو تدهور في إحدى وصلات شبكة SD-WAN ، وذلك من خلال تطبيق تأثيرات مثل فقدان الحزم أو زيادة التأخير أو تقليل جودة الوصلة. يستخدم النظام هذه العملية لاختبار قدرة آلية المعالجة الذاتية (Self-Healing) على اكتشاف العطل، وتحليل حالة الشبكة، واختيار مسار بديل مناسب، ثم إعادة توجيه حركة المرور والتحقق من نجاح عملية الاستعادة
الفاعلين Actors	مدير الشبكة (Network Administrator): يحدد الوصلة المستهدفة ونوع العطل وقيمه ومدته . النظام: ينفذ حقن العطل ويراقب تأثيره ويطلق آلية المعالجة الذاتية .
الشروط السابقة Precondition	- تشغيل شبكة SD-WAN ووجود الوصلة المستهدفة - تشغيل نظام المراقبة وجمع بيانات Telemetry . - توفر مسار أو أكثر يمكن استخدامه كمسار بديل عند الحاجة . - متلاك مدير الشبكة صلاحية تنفيذ سيناريو حقن العطل . - تشغيل مكونات المعالجة الذاتية المسؤولة عن اكتشاف العطل والاستجابة له.

الشروط اللاحقة Postcondition - تم تطبيق العطل المحدد على الوصلة المستهدفة خلال الفترة المطلوبة . - تم رصد تأثير العطل على مؤشرات الشبكة مثل Latency, Jitter, Packet Loss, Bandwidth . -تم اكتشاف حالة تدهور الوصلة أو فشلها وفق سياسات النظام . -إذا استدعى الأمر، تم اختيار مسار بديل والتحقق من سلامة قرار إعادة التوجيه . -تم تنفيذ عملية إعادة التوجيه بواسطة وحدة التنفيذ عبر وحدة التحكم Ryu -تم التحقق من حالة الشبكة بعد التعافي وتسجيل نتيجة عملية الـSelf Healing .	
--	--

الجدول 1: الوصف النصي لحالة الاستخدام inject link fault

سير الأحداث

السيناريو الأساسي الناجح

مدير الشبكة	النظام
1. يقوم باختيار وصلة و تحديد نوع العطل المراد تطبيقه و الضغط على زر التنفيذ .	
2. يتحقق النظام من صحة الوصلة المستهدفة ومن إمكانية تنفيذ سيناريو العطل .	
3. يقوم النظام بتطبيق العطل المحدد على الوصلة، مثل زيادة فقدان الحزم أو التأخير باستخدام آلية محاكاة الأعطال.	
4. تبدأ مكونات المراقبة بجمع قياسات أداء الوصلة والشبكة أثناء وجود العطل .	
5. يقوم النظام بتسجيل مؤشرات الشبكة مثل Latency و Jitter و Packet Loss و Bandwidth .	
6. يحلل النظام قياسات SLA للنق (latency, jitter, loss, bandwidth) المقاسة فعلياً عبر ping/iperf3 ، ويكتشف تدهوراً في حالة النق	

(WARNING/CRITICAL/UNREACHABLE)	وفقًا لعتبات محددة مسبقًا
7. يخضع النظام حالة النفق الجديدة لآلية تثبيت (Debounce)، فلا تُعتمد إلا بعد تكرارها لجولتي قياس متتاليتين على الأقل، لتفادي ردود الفعل على تذبذب لحظيو يقوم النظام عندها بتحديث الحالة الحالية للشبكة وإرسالها إلى مكونات اتخاذ القرار.	
8. يقوم نظام اتخاذ القرار بتقييم حالة الشبكة والمسارات البديلة المتاحة.	
9. يحدد النظام المسار البديل الأنسب وفقًا لمقاييس الشبكة وسياسات الـ SLA وآلية اختيار المسار.	
10. تقوم وحدة الحماية والتحقق (Policy & Safety / Guardrail) بالتحقق من صحة قرار إعادة التوجيه.	
11. تقوم وحدة التنفيذ بإرسال أمر إعادة التوجيه إلى وحدة التحكم، Ryu.	
12. يقوم Ryu Controller بتطبيق قواعد OpenFlow اللازمة لتحويل حركة المرور إلى المسار البديل.	
13. تستمر وحدة المراقبة في جمع القياسات للتحقق من حالة الشبكة بعد إعادة التوجيه.	
14. يتحقق النظام من تحسن أداء المسار واستعادة الاتصال وتحقيق متطلبات الـ SLA حيث بعد فترة استقرار، يقيس النظام التحسن الفعلي في SLA النفق نتيجة القرار، ويستخدمه لتحديث أوزان نموذج اتخاذ القرار (تعلم مستمر عبر التغذية الراجعة)	
15. سجل النظام نتائج عملية حقن العطل وعملية المعالجة الذاتية، بما في ذلك حالة الوصلة والمسار السابق والمسار الجديد ومؤشرات الأداء قبل وبعد التعافي.	

16. يراقب مدير الشبكة نتائج سيناريو العطل من خلال لوحة المراقبة.	
17. ينهي النظام سيناريو حقن العطل بنجاح ويعيد الوصلة إلى حالتها الطبيعية عند انتهاء مدة العطل.	

الجدول 2: سيناريو النجاح الخاص بحالة inject link fault

المسارات البديلة

1. انتهاء مدة العطل دون الحاجة إلى إعادة توجيه

في المرحلة 6، إذا أدى العطل إلى تدهور مؤقت ولم يتجاوز عتبات الفشل المحددة، يستمر النظام في مراقبة الوصلة دون تشغيل آلية إعادة التوجيه. وبعد انتهاء مدة العطل، يعيد النظام الوصلة إلى حالتها الطبيعية ويسجل نتائج القياس.

2. اكتشاف العطل مع توفر أكثر من مسار بديل

في المرحلة 8، إذا اكتشف النظام وجود عدة مسارات بديلة صالحة، يقوم بتقييمها ومقارنتها وفقاً لمقاييس الأداء وسياسات النظام، ثم يختار المسار صاحب أفضل تقييم.

مسارات الأخطاء :

E1: في المرحلة 2، الوصلة المستهدفة غير موجودة أو غير متاحة، يقوم النظام باظهار الخطأ وإلغاء عملية حقن العطل.

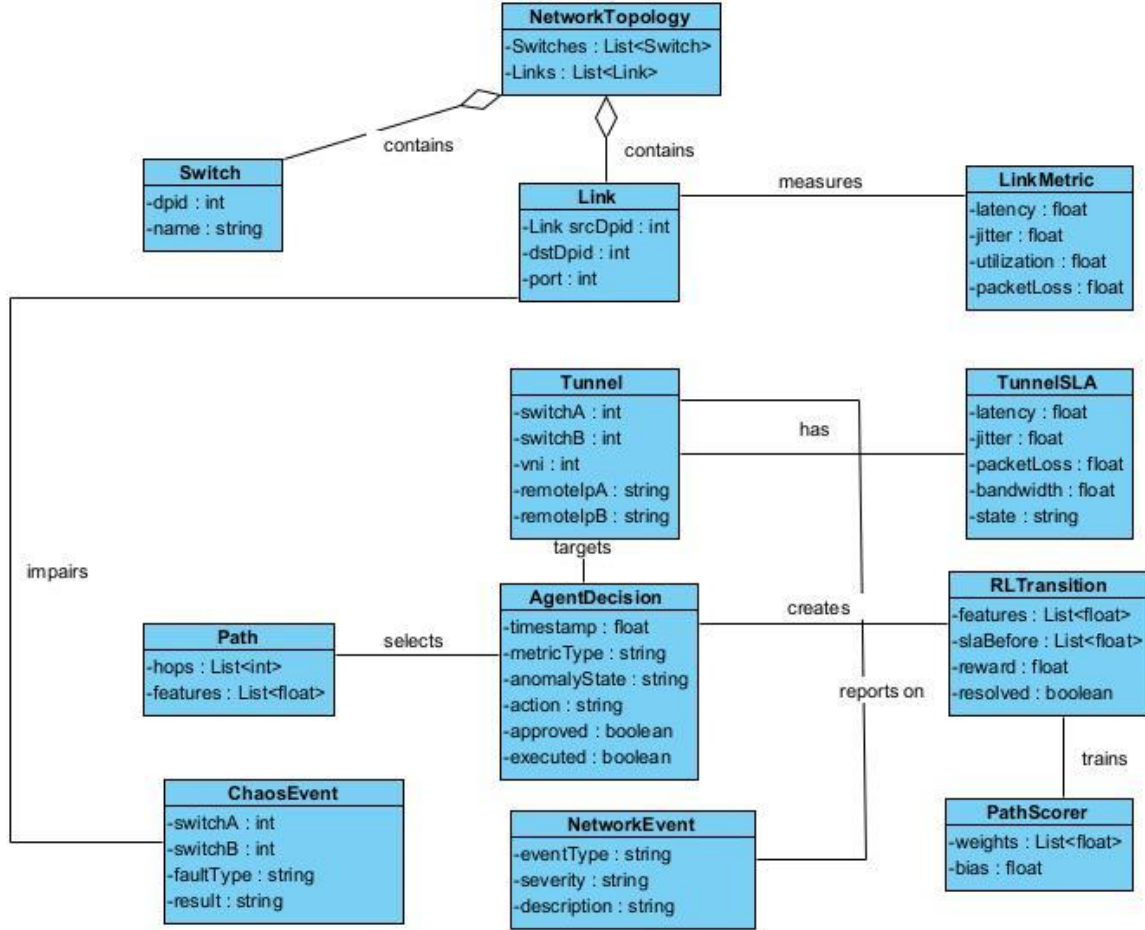
E2: في المرحلة 3، فشل تطبيق العطل على الوصلة. يقوم النظام بتسجيل الخطأ وإنهاء السيناريو.

E3: في المرحلة 9، لا يوجد مسار بديل صالح. يسجل النظام الحالة ويبلغ مدير الشبكة بفشل الاستعادة.

E4: في المرحلة 10، رفضت وحدة الحماية قرار إعادة التوجيه. يقوم النظام بإلغاء القرار وتسجيل سبب الرفض.

3.2- مخطط المفاهيم (مخطط الصفوف الأولي)

يبين هذا القسم مخططاً أولياً للمفاهيم الأساسية الموجودة في نظام Self-Healing and Self-Managed SD-WAN والعلاقات التي تربط بينها. يهدف المخطط إلى تحديد الكيانات والمفاهيم الرئيسية في النظام قبل الانتقال إلى التصميم التفصيلي للصفوف البرمجية.



الشكل 2: مخطط المفاهيم

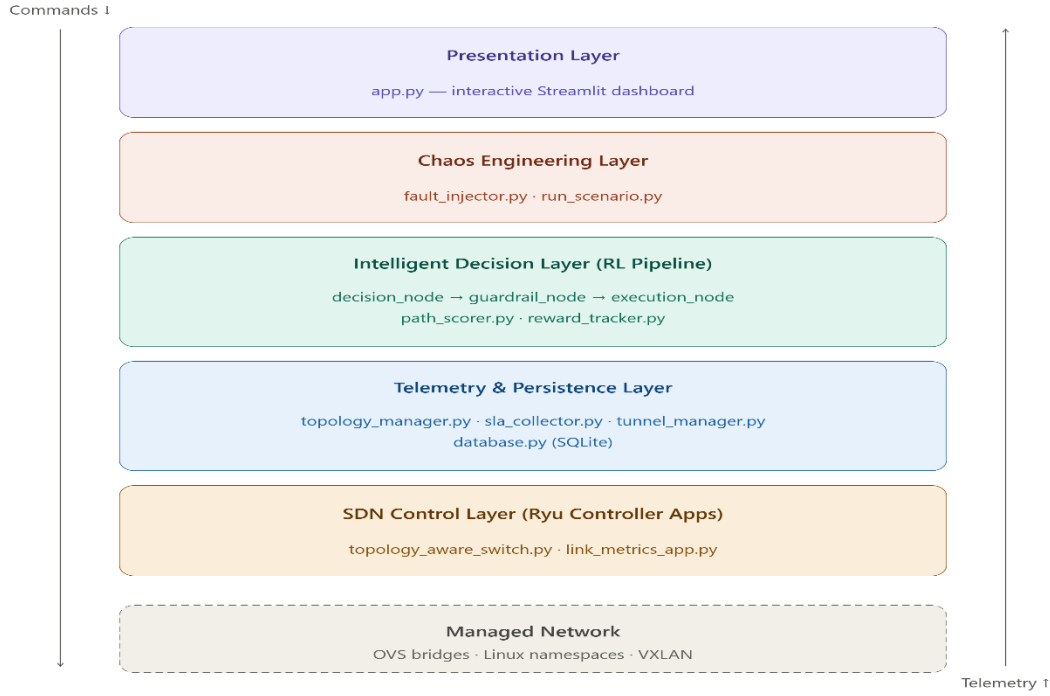
الفصل الثالث

1.3- التصميم الأولي للنظام

يعتمد التصميم الأولي لنظام Self-Healing SD-WAN على بنية طبقية تهدف إلى المراقبة المستمرة لأداء الأنفاق الفوقية VXLAN Tunnels واكتشاف حالات التدهور التي قد تؤثر في جودة الاتصال. يقوم النظام بصورة دورية بجمع مؤشرات أداء الأنفاق والمسارات الفيزيائية التي تمر من خلالها، مثل Latency و Jitter و Packet Loss و Bandwidth و Utilization، ثم يعالج هذه البيانات لبناء الحالة الحالية للشبكة. عند اكتشاف تدهور في أداء أحد الأنفاق يتجاوز الحدود أو السياسات المحددة، تبدأ آلية المعالجة الذاتية بتقييم المسارات الفيزيائية البديلة المتاحة في شبكة ال Underlay. يستخدم النظام آلية قائمة على Reinforcement Learning لاختيار الإجراء أو المسار الفيزيائي الأنسب، مع تمرير القرار عبر طبقة Policy & Safety / Guardrail للتحقق من صلاحيته قبل التنفيذ. يتم تنظيم مراحل سير العمل وانتقال حالة المعالجة باستخدام LangGraph، الذي يعمل كإطار لتنسيق العقد البرمجية وليس كنموذج لاتخاذ القرار بحد ذاته. بعد اعتماد القرار، تقوم طبقة التنفيذ بالتواصل مع Ryu Controller لتطبيق قواعد OpenFlow اللازمة على محولات OVS، مما يؤدي إلى تغيير المسار الفيزيائي في شبكة ال Underlay الذي تمر عبره حركة النفق، مع الحفاظ على النفق الفوقية نفسه. بعد تنفيذ عملية إعادة التوجيه، يستمر النظام في مراقبة النفق للتحقق من تحسن مؤشرات الأداء وتحقيق متطلبات ال SLA، وبذلك تشكل العملية حلقة تحكم مغلقة من المراقبة → اكتشاف التدهور → اختيار المسار الفيزيائي البديل → التحقق → التنفيذ → التحقق من التعافي .

الطبقات المعمارية العامة:

يتكون النظام بشكل أساسي من الطبقات التالية :



الشكل 3: الطبقات المكونة للنظام

و يوضح الجدول التالي المكونات الأساسية لكل طبقة و وظيفتها .

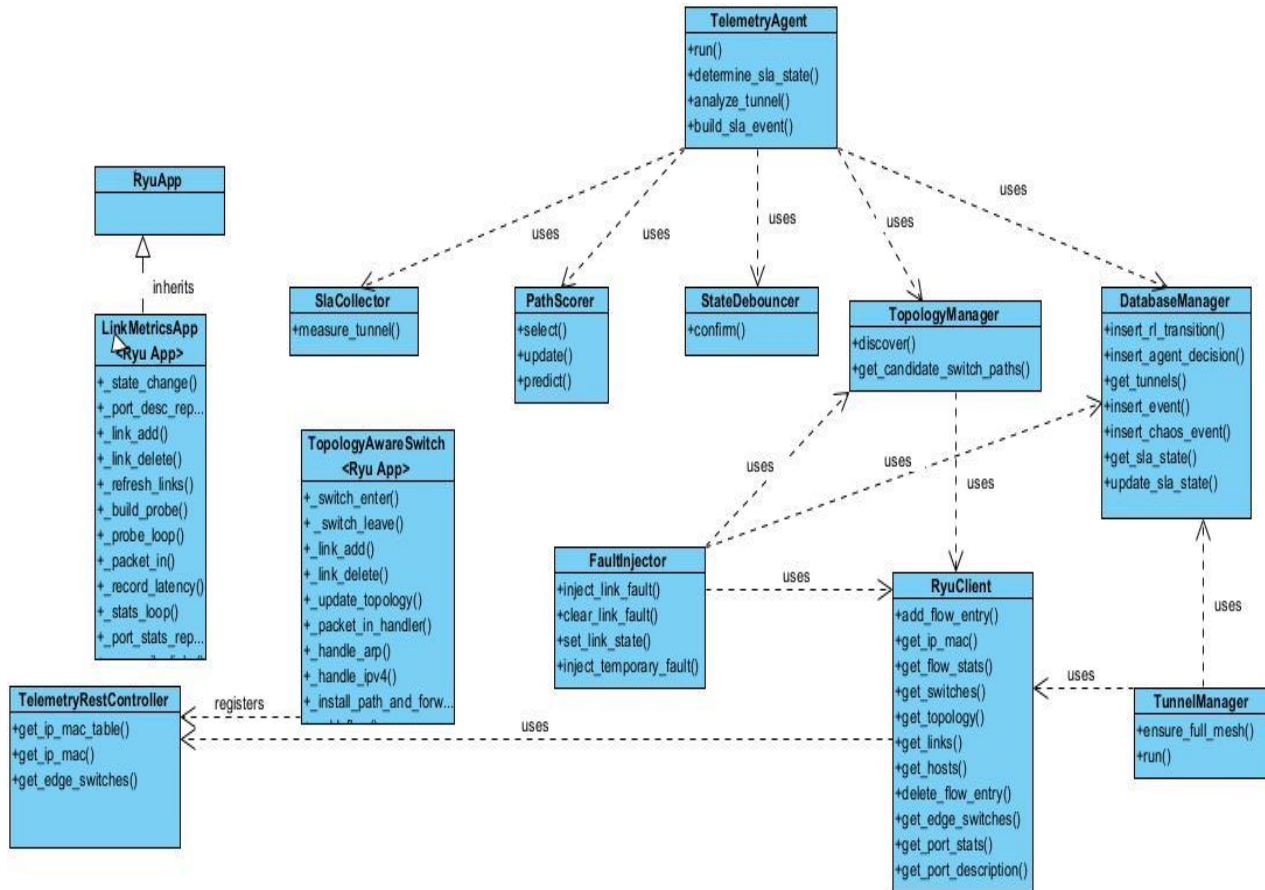
الوظيفة	المكونات	الطبقة
تجسيد البنية التحتية الفعلية (سويتشات، وصلات، أنفاق) التي تُنفَّذ عليها كل قرارات الطبقات العليا	Containernet, Open vSwitch, Linux network namespaces	طبقة البنية التحتية (Managed Network)
التحكّم الفعلي بمستوى البيانات: توجيه الحزم، وقياس أداء الروابط الفيزيائية مباشرة من داخل المتحكم .	topology_aware_switch.py, link_metrics_app.py (Ryu apps)	طبقة تحكّم (SDN (Ryu Controller Apps)
جمع صورة حيّة وموثوقة عن حالة الشبكة (طوبولوجيا، أداء الأنفاق) وإدارة طبقة التراكب VXLAN ، مع الاحتفاظ بسجل دائم لكل الأحداث والقرارات.	topology_manager.py, sla_collector.py, state_debouncer.py, ryu_client.py, database.py, config.py	طبقة القياس والاستمرارية (Telemetry & Persistence)
تحويل مؤشرات تدهور الأداء إلى قرار إعادة توجيه آمن ومُتحقّق منه، وتعلّم تحسين هذا القرار تدريجيًا من نتائج الفعلية على الشبكة	agents/graph.py, decision_node.py, guardrail_node.py, execution_node.py, active_path.py,	طبقة القرار الذكي (Intelligent Decision — RL)

	metrics_provider.py, path_scorer.py, reward_tracker.py, telemetry_agent.py	
محاكاة أعطال شبكة حقيقية (تأخير، فقد حزم، انقطاع) بشكل متحكم به، لاختبار قدرة النظام على الاكتشاف والاستجابة التلقائية	fault_injector.py, run_scenario.py	طبقة هندسة الفوضى (Chaos Engineering)
توفير واجهة تفاعلية واحدة لمراقبة حالة الشبكة والوكيل الذكي حيًا، والسماح بالتدخل اليدوي (حقن الأعطال، مراجعة السجلات) دون المساس بمنطق النظام الداخلي	dashboard/app.py (Streamlit)	طبقة العرض (Presentation)

الجدول 3: الطبقات المكونة للنظام و وظائفها

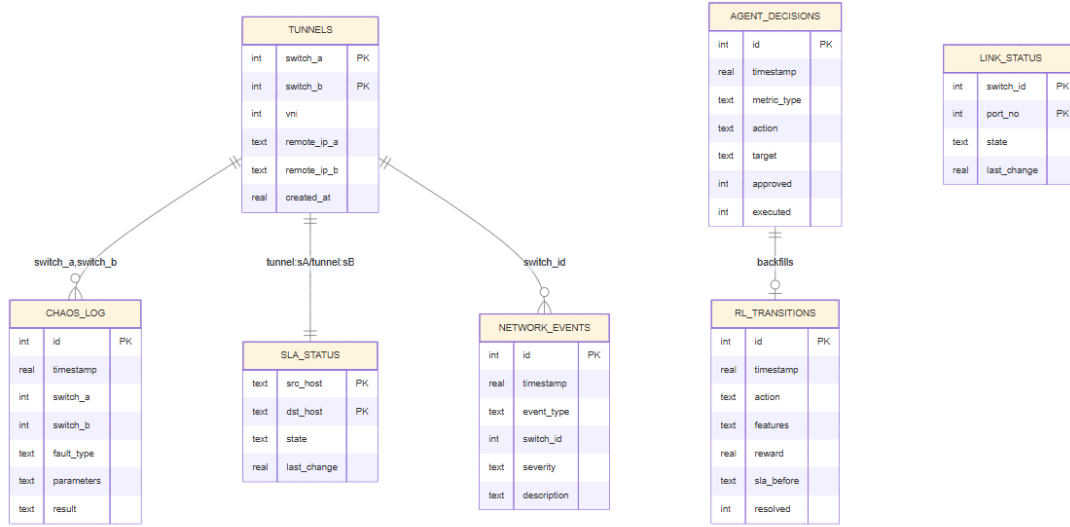
2.3- التصميم التفصيلي

1.2.3- مخطط الصفوف التصميمي



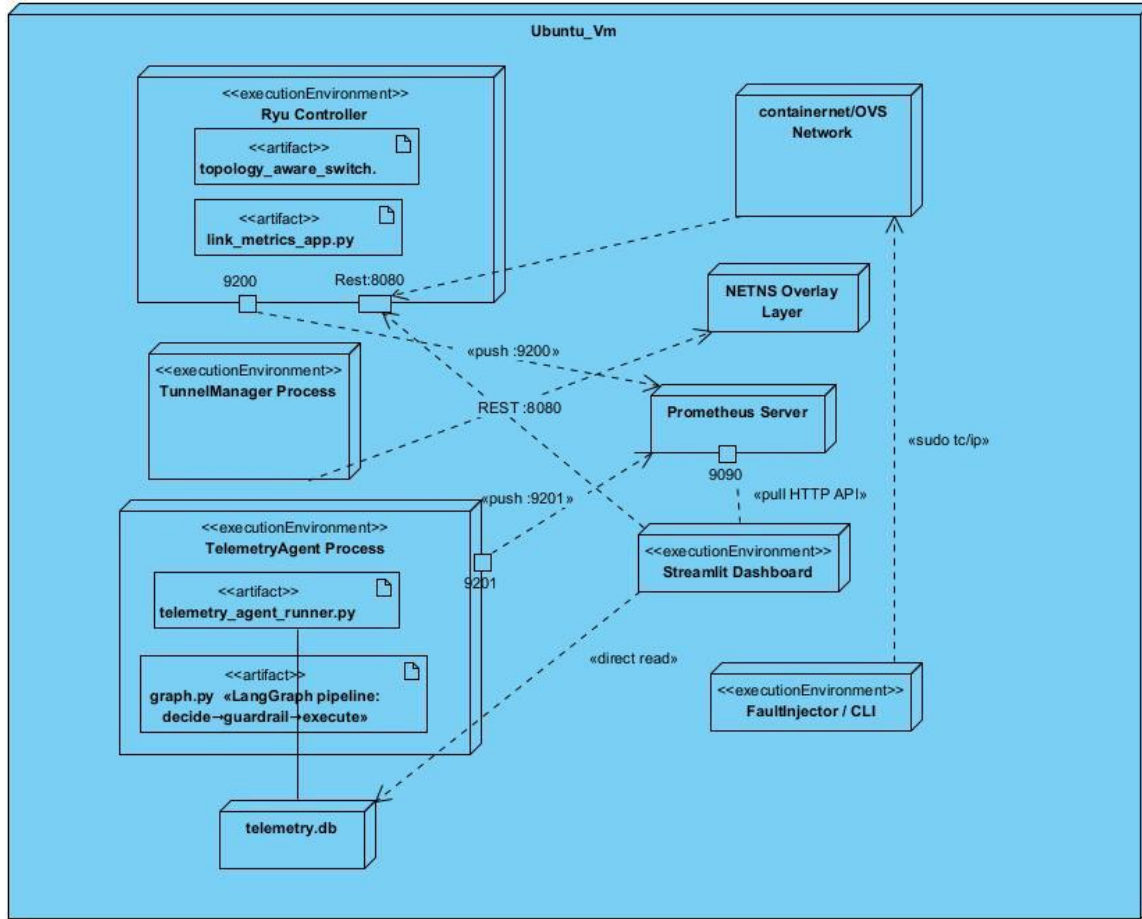
الشكل 4: مخطط الصفوف التفصيلي

2.2.3- تصميم قاعدة المعطيات



الشكل 5: تصميم قاعدة المعطيات

3.2.3- تصميم البنية الفيزيائية



الشكل 6: تصميم البنية الفيزيائية

4.2.3- تصميم الخوارزميات (الإجرائيات Processes)

يحدد هذا القسم الخوارزميات الأساسية التي تُنفذ السلوك المستقل للنظام، معبراً عنها بلغة شبه رمزية منظمة. توضح كل خوارزمية غرضها، ومدخلاتها/مخرجاتها، والمنطق الإجرائي المطبق فعلياً في قاعدة التعليمات البرمجية.

Algorithm 1 — SLA State Classification:

Purpose: Classify a tunnel's measured end-to-end metrics into a discrete health state.

Source: telemetry_agent.determine_sla_state()

Input: metrics = {latency_ms, jitter_ms, packet_loss_percent, bandwidth_Mbps}

Output: state $\in$ {NORMAL, WARNING, CRITICAL, UNREACHABLE}

ALGORITHM DetermineSlaState(metrics)

BEGIN

```

    IF metrics.latency = NULL AND metrics.loss = NULL AND metrics.bandwidth =
    NULL THEN
        RETURN UNREACHABLE          // no probe response at all

    FUNCTION Breaches(lossLimit, latencyLimit, jitterLimit, bwFloor)
        RETURN (metrics.loss  ≠ NULL AND metrics.loss  ≥ lossLimit)
            OR (metrics.latency ≠ NULL AND metrics.latency ≥ latencyLimit)
            OR (metrics.jitter  ≠ NULL AND metrics.jitter ≥ jitterLimit)
            OR (metrics.bandwidth ≠ NULL AND metrics.bandwidth ≤ bwFloor)

    IF Breaches(LOSS_CRITICAL, LATENCY_CRITICAL, JITTER_CRITICAL,
    BW_MIN_CRITICAL) THEN
        RETURN CRITICAL
    IF Breaches(LOSS_WARNING, LATENCY_WARNING, JITTER_WARNING, BW_MIN_WARNING)
    THEN
        RETURN WARNING
    RETURN NORMAL
END

```

Complexity: $O(1)$ — constant number of threshold comparisons.

Algorithm 2 — State Debouncing:

Purpose: Suppress single-sample noise; only accept a state change after it repeats N consecutive rounds.

Source: StateDebouncer.confirm()

Input: key (tunnel identity), candidateState

Output: confirmed state, or NULL if not yet confirmed .

```

ALGORITHM Confirm(key, candidateState)
BEGIN
    LOCK(mutex)
    (pendingState, streak) ← pendingTable[key] OR (NULL, 0)

    IF candidateState = pendingState THEN
        streak ← streak + 1
    ELSE
        pendingState ← candidateState
        streak ← 1

    pendingTable[key] ← (pendingState, streak)
    UNLOCK(mutex)

    IF streak ≥ CONFIRM_READS THEN
        RETURN pendingState
    ELSE
        RETURN NULL
END

```

Note: Thread-safe — invoked concurrently from parallel probe threads (ThreadPoolExecutor).

Algorithm 3 — RL Path Selection (ϵ -greedy):

Purpose: Choose a rerouting action among candidate paths, balancing exploitation of the learned model with exploration.

Source: PathScorer.select()

Input: options[] (each with features or NULL for the NONE baseline), epsilon

Output: (chosenOption, score, wasExploration)

```
ALGORITHM Select(options, epsilon)
BEGIN
  IF NONE not present in options THEN
    APPEND options, {action: NONE, features: NULL}

  FOR EACH option IN options DO
    IF option.features = NULL THEN
      scores[option]  $\leftarrow$  NONE_BASELINE_SCORE
    ELSE
      scores[option]  $\leftarrow$  Predict(option.features) //  $w \cdot x + b$ 

  IF Random(0,1) < epsilon THEN
    chosen  $\leftarrow$  RandomChoice(options) // exploration
    wasExploration  $\leftarrow$  TRUE
  ELSE
    bestScore  $\leftarrow$  MAX(scores)
    candidates  $\leftarrow$  { o : scores[o] = bestScore }
    chosen  $\leftarrow$  RandomChoice(candidates) // random tie-break
    wasExploration  $\leftarrow$  FALSE

  RETURN (chosen, scores[chosen], wasExploration)
END
```

Design rationale: Random tie-breaking prevents the cold-start weights (near-zero) from deterministically favoring option 0 (the current shortest path) on every run.

Algorithm 4 — RL Weight Update (Online SGD):

Purpose: Adjust the linear model's weights after a reward is observed.

Source: PathScorer.update()

Input: features[4], reward, learningRate

Output: updated (weights, bias)

```
ALGORITHM Update(features, reward, learningRate)
```

```

BEGIN
    predicted ← Predict(features)
    error ← reward - predicted
    error ← CLIP(error, -ERROR_CLIP, +ERROR_CLIP)

    FOR i ← 0 TO FEATURE_SIZE - 1 DO
        weights[i] ← weights[i] + learningRate × error × features[i]
        weights[i] ← CLIP(weights[i], -WEIGHT_CLIP, +WEIGHT_CLIP)

    bias ← bias + learningRate × error
    bias ← CLIP(bias, -WEIGHT_CLIP, +WEIGHT_CLIP)
END

```

Design rationale: Design rationale: Double-clipping (error, then weights) is a defense-in-depth measure for avoiding weight explosion

Algorithm 5 — Guardrail Validation:

Purpose: Independently verify a proposed reroute against the live network before allowing execution.

Source: guardrail_node._check_reroute()

Input: target = {switchA, switchB, selectedPath}, candidatePaths[], metricType

Output: (approved: bool, reason: string)

```

ALGORITHM CheckReroute(target, candidatePaths, metricType)
BEGIN
    IF target.switchA = NULL OR target.switchB = NULL THEN
        RETURN (FALSE, "invalid target identity")

    IF candidatePaths = EMPTY THEN
        RETURN (FALSE, "no candidates to verify against")    // fail closed

    IF target.selectedPath = NULL THEN
        RETURN (FALSE, "no selected_path in target")

    IF target.selectedPath NOT IN candidatePaths THEN
        RETURN (FALSE, "selected_path is stale")

    activePath ← GetActivePath(metricType, target, candidatePaths)

    IF target.selectedPath = activePath THEN
        RETURN (FALSE, "already the active path – no-op")

    RETURN (TRUE, "verified")
END

```

Algorithm 6 — Reward Computation:

Purpose: Score a reroute's real-world effect by comparing tunnel SLA before vs. after, with a settle delay.

Source: reward_tracker._reward_from_sla()

Input: before[3], after[3] — each [latency, jitter, loss] normalized to [0,1] or NULL

Output: scalar reward, or NULL if nothing comparable

```
ALGORITHM RewardFromSla(before, after)
BEGIN
  weights ← [W_LATENCY, W_JITTER, W_LOSS]
  terms ← EMPTY LIST

  FOR i ← 0 TO 2 DO
    IF before[i] ≠ NULL AND after[i] ≠ NULL THEN
      APPEND terms, weights[i] × (before[i] - after[i])

  IF terms = EMPTY THEN
    RETURN NULL

  reward ← SUM(terms)

  wasUnreachable ← (before[LOSS] ≠ NULL AND before[LOSS] ≥ 1.0)
  isReachable ← (after[LOSS] ≠ NULL AND after[LOSS] < 1.0)
  IF wasUnreachable AND isReachable THEN
    reward ← reward + UNREACHABLE_RECOVERY_BONUS

  RETURN CLIP(reward, -REWARD_CLIP, +REWARD_CLIP)
END
```

Design rationale: Terms with a missing side are skipped rather than defaulted to 0 — treating a lost probe as "perfect health" would poison the training signal.

Algorithm 7 — VXLAN Full-Mesh Reconciliation:

Purpose: Keep the overlay tunnel mesh consistent with the current set of edge switches, on every scheduler tick.

Source: tunnel_manager.ensure_full_mesh()

Input: none (reads controller state)

Output: side effect — tunnels created/torn down

```
ALGORITHM EnsureFullMesh()
BEGIN
  edgeDpids ← DiscoverEdgeSwitches()           // from controller REST

  FOR EACH dpid IN edgeDpids DO
    EnsureEndpoint(dpid)                       // VTEP + namespace + veth
```

```

        SendGratuitousArp(dpid)

existingPairs ← { (a,b) : (a,b) IN Tunnels() }
wantedPairs  ← { (a,b) : a,b IN edgeDpids, a ≠ b }

FOR EACH (a,b) IN (wantedPairs - existingPairs) DO
    CreateTunnel(a, b)

FOR EACH (a,b) IN existingPairs DO
    IF a ∉ edgeDpids OR b ∉ edgeDpids THEN
        RemoveTunnel(a, b)                // stale tunnel to a departed
edge
END

```

Algorithm 8 — Active Path Reconstruction:

Purpose: Determine which path a tunnel's traffic is *actually* using right now, preferring a live read over history.

Source: active_path.get_active_path()

Input: metricType, target, candidatePaths[], topology

Output: path[] or NULL

```

ALGORITHM GetActivePath(metricType, target, candidatePaths, topology)
BEGIN
    livePath ← ReconstructLiveTunnelPath(target, topology)
    // walk override flows
    IF livePath ≠ NULL THEN
        RETURN livePath

    // fallback: last executed REROUTE for this identity
    FOR EACH decision IN RecentRerouteDecisions(metricType) DO
        IF TargetIdentity(decision.target) = TargetIdentity(target) THEN
            IF decision.target.selectedPath IN candidatePaths THEN
                RETURN decision.target.selectedPath
            BREAK                // newest is already stale

    RETURN candidatePaths[0]    // never rerouted ⇒ default shortest path
END

SUBROUTINE ReconstructLiveTunnelPath(target, topology)
BEGIN
    path ← [target.switchA]; current ← target.switchA; visited ← {current}
    FOR i ← 1 TO MAX_LIVE_PATH_HOPS DO
        IF current = target.switchB THEN RETURN (path, COMPLETE)
        outPort ← OverrideFlowOutputPort(current, dstMac)
        IF outPort = NULL THEN RETURN (path, INCOMPLETE)    // chain ends
early
        next ← NeighborBehindPort(topology, current, outPort)
        IF next = NULL OR next IN visited THEN RETURN (path, INCOMPLETE)
        APPEND path, next; visited ← visited U {next}; current ← next
    RETURN (path, INCOMPLETE)    // loop guar exceeded
END

```

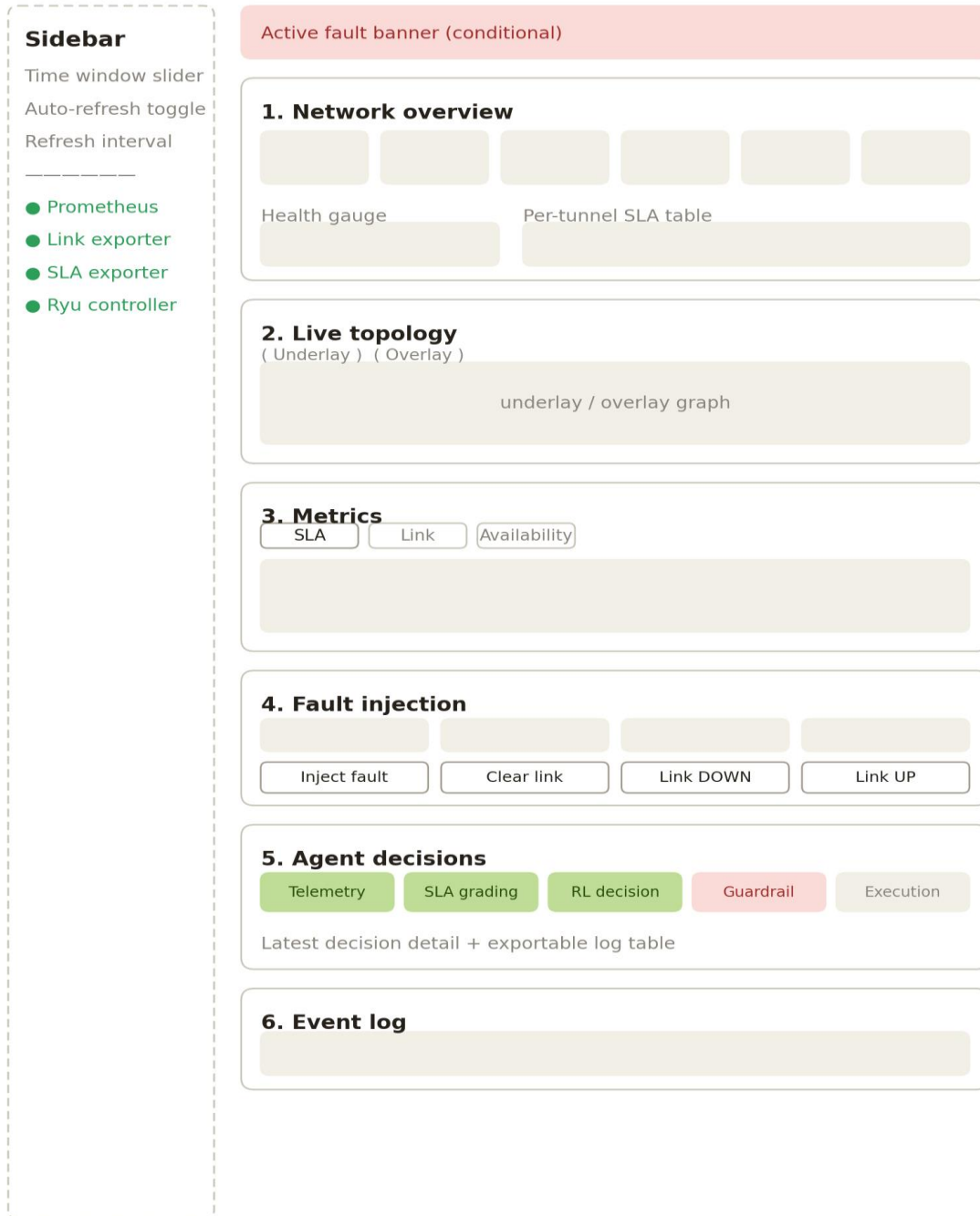
5.2.3- تصميم الواجهات

الواجهة الوحيدة في النظام هي لوحة التحكم التفاعلية المبنية بـ (Streamlit (app.py وهي صفحة واحدة (Single-Page Dashboard) مقسّمة إلى 7 أقسام رأسية، بالإضافة إلى شريط تحكم جانبي ثابت.

ملاحظات تصميمية مهمة:

- نمط تفاعل واحد مهيم (`st.rerun()`: أي فعل (حقن عطل، تحديث الفلتر) يعيد تحميل الصفحة كاملة بدل تحديث جزئي (AJAX) و هذا قيد أساسي بسبب استخدام Streamlit .
- التخزين المؤقت (`@st.cache_data(ttl=8)`: على استعلامات Prometheus يعني أن الواجهة ليست "لحظية 100%" بل بتأخير أقصاه 8 ثوانٍ، وهذا قرار تصميمي متعمّد لتفادي إغراق Prometheus بطلبات متكررة .

مخطط توزيع الشاشة (Wireframe) :



الشكل 7: مخطط توزيع الشاشة الخاص بواجهة المراقبة و التحكم

الفصل الرابع

التنفيذ والاختبارات

Implementation & Testing

نقدم في هذا الفصل شرحاً عن بيئة العمل المستخدمة في تنجيز النظام مع تبين الأدوات المستعملة. كما نبين خطة الاختبارات التي تم اعتمادها في التحقق من صحة التطبيق مع تبين بعض الأمثلة من التطبيق. كما نبين الخلاصة وبعض الآفاق المستقبلية.

1.4- بيئة العمل وأدوات التنفيذ

1. بيئة التشغيل :

العنصر	التفصيل
نظام التشغيل	Linux (Ubuntu)
لغة البرمجة	Python 3
محاكي/بنية الشبكة	containernet
المتحكم SDN	اطار عمل (ryu-manager) Ryu

الجدول 4: بيئة التشغيل الخاصة بالنظام

2. المكتبات والأطر البرمجية المستخدمة

الفئة	الأداة/المكتبة	الاستخدام في المشروع
SDN Framework	ryu	تطوير تطبيقات التحكم (topology_aware_switch.py, link_metrics_app.py)
تمثيل الرسوم البيانية	networkx	بناء طوبولوجيا الشبكة وحساب المسارات البديلة (shortest_simple_paths)
التعلم المعزز/سير العمل	langgraph	بناء decide → guardrail → execute

قاعدة البيانات	sqlite3 (WAL mode)	تخزين القرارات، الأحداث، سجل الأعطال، بيانات التدريب
المراقبة والقياس	prometheus_client + خادم Prometheus	تجميع مقاييس الروابط والأنفاق الزمنية
لوحة التحكم	streamlit + plotly + pandas	الواجهة التفاعلية
الشبكات الافتراضية	VXLAN	طبقة التراكب (Overlay)
قياس الأداء	ping, iperf3	قياس SLA الفعلي (زمن استجابة، فقد، عرض نطاق)
اتصال REST	requests, webob (Ryu WSGI)	التواصل بين طبقة القياس والمتحكم
هندسة الفوضى	tc netem, ip link	حقن أعطال التأخير/الفقد/الانقطاع

2.4- الاختبارات والأمثلة :

تم تصميم النظام ليكون قابلاً للتكيف والتوسع مع التغيرات في طوبولوجيا الشبكة، بحيث يستطيع اكتشاف الشبكة والتعرف تلقائياً على مكوناتها والأجهزة الموجودة فيها، وتحديد الروابط بينها دون الحاجة إلى ضبطها يدوياً. وللتحقق من هذه القدرة، تم اختبار النظام على طوبولوجيات مختلفة، شملت طوبولوجيا مكونة من 4 محولات (Switches) وأخرى مكونة من 8 محولات، مع التأكد من قدرته على اكتشاف الطوبولوجيا والتعامل معها بشكل صحيح ، تم كذلك محاكاة حالات مختلفة لحقن الأعطال في الشبكة على كلتا الطوبولوجيتين، بهدف التحقق من قدرة النظام على اكتشاف الأعطال وتحديد حالتها، ثم اتخاذ قرار مناسب لإعادة توجيه حركة المرور عبر مسار بديل، مما يوضح قدرة النظام على التكيف مع الأعطال والتعامل معها بشكل مستقل .

1. الاختبار على الطوبولوجيا الأولى (4 محولات):

نبدأ أولاً بتشغيل متحكم Ryu مع تطبيق التوجيه الذي تم تصميمه، بالإضافة إلى تطبيق مراقبة وقياس مؤشرات أداء الوصلات، المسؤول عن جمع وحساب المؤشرات اللازمة لكل وصلة في الشبكة، مثل زمن التأخير (Latency) ،

والتذبذب (Jitter) ، ونسبة فقدان الحزم (Packet Loss) ، واستخدام عرض النطاق الترددي (Bandwidth Utilization). كما يتم تشغيل واجهات REST API الخاصة بمتحكم Ryu ، والمتمثلة في

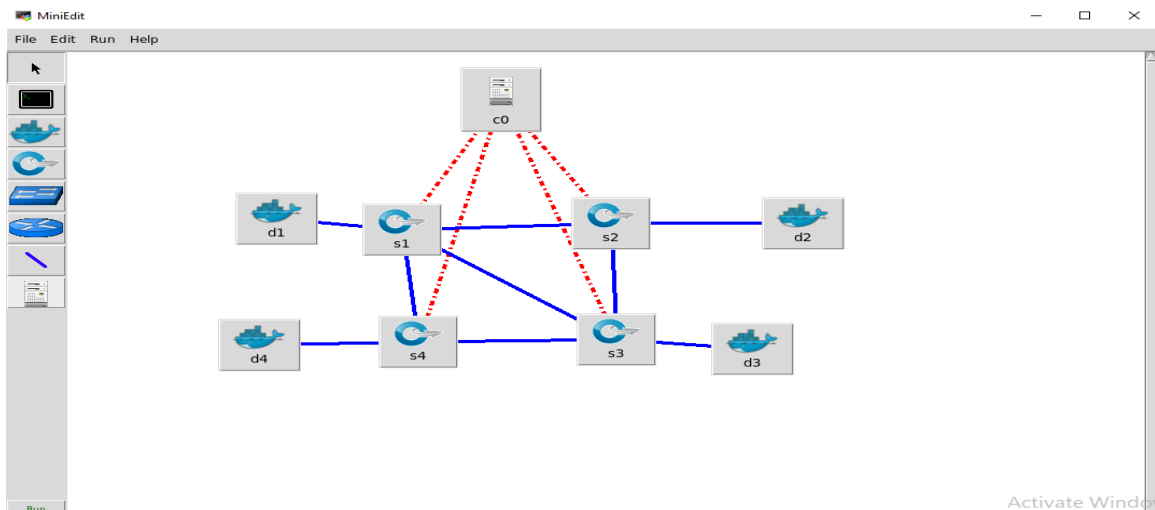
ryu.app.rest_topology و ryu.app.ofctl_rest ، وذلك لإتاحة الوصول البرمجي إلى معلومات الشبكة وإحصائيات المنافذ والطوبولوجيا، وتمكين مكونات النظام الأخرى من الاستفادة منها أثناء عملية المراقبة واتخاذ القرارات :

```
root@shouib:/home/shouib/SD-WAN# ryu-manager --observe-links controller/topology_aware_switch.py controller/link_metrics_app.py
ryu.app.ofctl_rest ryu.app.rest_topology
loading app controller/topology_aware_switch.py
loading app controller/link_metrics_app.py
loading app ryu.app.ofctl_rest
loading app ryu.app.rest_topology
loading app ryu.controller.ofp_handler
loading app ryu.topology.switches
loading app ryu.controller.ofp_handler
creating context wsgi
instantiating app None of DPSet
creating context dpset
instantiating app controller/topology_aware_switch.py of TopologyAwareSwitch
instantiating app controller/link_metrics_app.py of LinkMetricsApp
[link_metrics] /metrics exposed on :9200
instantiating app ryu.app.ofctl_rest of RestStatsApi
instantiating app ryu.app.rest_topology of TopologyAPI
instantiating app ryu.controller.ofp_handler of OFPHandler
instantiating app ryu.topology.switches of Switches
(2865) wsgi starting up on http://0.0.0.0:8080
```

بعد ذلك، نقوم بتشغيل Containernet/Miniedit، ثم اختيار الطوبولوجيا الأولى المكوّنة من أربعة محولات (Switches)، مع الأجهزة الطرفية والروابط المحددة فيها، ومن ثم تشغيل الطوبولوجيا لبدء محاكاة الشبكة وربطها بمتحكم Ryu. وبمجرد تشغيل الطوبولوجيا، يبدأ المتحكم باكتشاف مكونات الشبكة وربطها، بينما يقوم تطبيق القياس بجمع مؤشرات الأداء الخاصة بالوصلات.

```
shouib@shouib:~/containernet/examples$ sudo -E python3 miniedit.py
[sudo] password for shouib:
/home/shouib/containernet/examples/miniedit.py:24: DeprecationWarning: The distutils package is deprecated and slated for removal
in Python 3.12. Use setuptools or check PEP 632 for potential alternatives
  from distutils.version import StrictVersion
topo=None
```

ثم تفتح لنا واجهة miniedit الرسومية و نقوم باختيار الطوبولوجيا التي قمنا بتصميمها مسبقا و ربطها بالمتحكم:



الشكل 8: الطوبولوجيا 1

بعد تشغيل الشبكة، نلاحظ أن متحكم Ryu تمكن من اكتشاف طوبولوجيا الشبكة بشكل صحيح، وذلك من خلال تطبيق التوجيه والاكتشاف topology_aware_switch الذي تم تطويره ضمن النظام. حيث يقوم التطبيق بالتعرف على المحولات والروابط بينها وبناء تمثيل للطوبولوجيا الفعلية للشبكة، مما يتيح للمتحكم استخدامها لاحقاً في عمليات التوجيه واتخاذ قرارات إعادة التوجيه عند حدوث الأعطال .

```
[topology] 1 switches, 0 directed links
[topology] 2 switches, 0 directed links
[topology] 2 switches, 1 directed links
[topology] 3 switches, 2 directed links
[topology] 3 switches, 2 directed links
[topology] 4 switches, 4 directed links
[topology] 4 switches, 8 directed links
[topology] 4 switches, 8 directed links
[topology] 4 switches, 10 directed links
[topology] 4 switches, 10 directed links
[topology] 4 switches, 10 directed links
[topology] 4 switches, 10 directed links
[topology] 4 switches, 10 directed links
[topology] 4 switches, 10 directed links
```

المقصود بعبارة 10 Direct Links هو عدد الوصلات الفيزيائية المباشرة بين المحولات فقط، دون احتساب اتجاه حركة البيانات. أما تطبيق التوجيه والاكتشاف topology_aware_switch الذي تم تطويره، فيتعامل مع كل وصلة فيزيائية على أنها وصلتان موجهتان (Directed Links) ، واحدة في كل اتجاه بين المحولين. لذلك، فإن عدد الوصلات التي يعتمد عليها التطبيق في تمثيل الطوبولوجيا يكون ضعف عدد الوصلات الفيزيائية المباشرة

ونلاحظ من خلال واجهة Containernet نجاح تطبيق التوجيه في تمرير الحزم عبر المسارات المحددة، مما أدى إلى إنشاء اتصال ناجح بين جميع الأجهزة الطرفية (Hosts) ضمن الطوبولوجيا. ويؤكد ذلك أن آلية اكتشاف الطوبولوجيا والتوجيه تعمل بالشكل المطلوب، وأن المحولات قادرة على توجيه حركة المرور بين مختلف الـ Hosts.

```
*** Starting CLI:
containernet> pingall
*** Ping: testing ping reachability
d1 -> d2 d3 d4
d2 -> d1 d3 d4
d3 -> d1 d2 d4
d4 -> d1 d2 d3
*** Results: 0% dropped (12/12 received)
containernet>
```

بعد ذلك، نقوم بتشغيل سكريبت بناء الأنفاق (VXLAN Tunnel Builder)، حيث يقوم السكريبت تلقائياً باكتشاف جميع Edge WAN Switches الموجودة في الطوبولوجيا، ثم إنشاء أنفاق VXLAN بينها وفقاً للطوبولوجيا المكتشفة. وبذلك نلاحظ نجاح النظام في اكتشاف جميع المحولات الطرفية وإنشاء الأنفاق بينها بشكل صحيح، دون الحاجة إلى تعريفها يدوياً..

```
shouib@shouib:~/SD-WAN$ sudo python3 -m telemetry.tunnel_manager
[sudo] password for shouib:
Using database: /home/shouib/SD-WAN/database/telemetry.db
[tunnel] TunnelManager started
[tunnel] edge switches: [1, 2, 3, 4]

[tunnel] creating VXLAN s1<->s2
[tunnel] CREATED s1<->s2 logical-vni=11002 directional-vni=11002/12001
[tunnel] creating VXLAN s1<->s3
[tunnel] CREATED s1<->s3 logical-vni=11003 directional-vni=11003/13001
[tunnel] creating VXLAN s1<->s4
[tunnel] CREATED s1<->s4 logical-vni=11004 directional-vni=11004/14001
[tunnel] creating VXLAN s2<->s3
[tunnel] CREATED s2<->s3 logical-vni=12003 directional-vni=12003/13002
[tunnel] creating VXLAN s2<->s4
[tunnel] CREATED s2<->s4 logical-vni=12004 directional-vni=12004/14002
[tunnel] creating VXLAN s3<->s4
[tunnel] CREATED s3<->s4 logical-vni=13004 directional-vni=13004/14003
```

بعد ذلك، نقوم بتشغيل تطبيق المراقبة وجمع القياسات، والمسؤول عن مراقبة أنفاق VXLAN وقياس مؤشرات الأداء الخاصة بها، مثل زمن التأخير (Latency)، والتذبذب (Jitter)، ونسبة فقدان الحزم (Packet Loss)، وعرض النطاق الترددي (Bandwidth). وتستخدم هذه القياسات لتحديد الحالة التشغيلية للأنفاق واكتشاف حالات التدهور أو الأعطال. وعند اكتشاف حالة تستدعي التدخل، يقوم التطبيق بإطلاق Event يؤدي إلى تشغيل بنية LangGraph، التي تتولى تحليل الحالة واتخاذ قرار إعادة توجيه حركة المرور عبر مسار بديل.

```
shouib@shouib:~/SD-WAN$ python3 -m agents.telemetry_agent_runner
Using database: /home/shouib/SD-WAN/database/telemetry.db
[sla_collector] /metrics exposed on :9201
```

ثم نقوم بمحاكاة حالة حقن عطل بنسبة فقدان حزم (Packet Loss) تبلغ 20% على الوصلة المستهدفة، وهي نسبة تتجاوز عتبة فقدان الحزم (Threshold) المحددة في النظام، والتي تبلغ 5%. وبناءً على ذلك، يتوجب على نظام المراقبة اكتشاف تدهور حالة النفق بين S1 و S2 وجميع الأنفاق التي تمر بالوصلة المستهدفة وإطلاق الأحداث (Events) المرتبطة بهذه الأنفاق. بعد ذلك، يتم تشغيل بنية LangGraph لتحليل الحالة، حيث يقوم النظام بتقييم المسارات البديلة المتاحة باستخدام نموذج التعلم المعزز (RL)، ومن ثم اتخاذ القرار المناسب، سواء بإعادة توجيه حركة المرور عبر مسار بديل أو الإبقاء على المسار الحالي إذا كان التقييم يشير إلى أن إعادة التوجيه ليست الخيار الأفضل.

```
shouib@shouib:~/SD-WAN$ python3 chaos/run_scenario.py --link 1 2 --loss 20 --duration 60
Using database: /home/shouib/SD-WAN/database/telemetry.db
[tc] sudo tc qdisc replace dev s1-eth1 root netem loss 20.0%
[tc] sudo tc qdisc replace dev s2-eth1 root netem loss 20.0%
```

فلاحظ أن النظام قام بإطلاق الأحداث الخاصة بأنفاق VXLAN المتأثرة و تقديم الاستجابة المطلوبة للتعافي .

```
shouib@shouib:~/SD-WAN$ python3 -m agents.telemetry_agent_runner
Using database: /home/shouib/SD-WAN/database/telemetry.db
[sla_collector] /metrics exposed on :9201
[TUNNEL SLA EVENT] s1 <-> s2 NORMAL -> CRITICAL
[GRAPH] tunnel state: CRITICAL action: NONE approved: False executed: None detail: No action was proposed
[TUNNEL SLA EVENT] s2 <-> s4 NORMAL -> CRITICAL
[EXECUTE][REROUTE] OK: Rerouted tunnel s2<->s4 onto ['s2', 's3', 's4'] (priority=100, idle_timeout=300s), forward on ['s2', 's3'],
reverse on ['s4', 's3']
[GRAPH] tunnel state: CRITICAL action: REROUTE approved: True executed: True detail: Rerouted tunnel s2<->s4 onto ['s2', 's3', 's
4'] (priority=100, idle_timeout=300s), forward on ['s2', 's3'], reverse on ['s4', 's3']
[TUNNEL SLA EVENT] s2 <-> s4 CRITICAL -> NORMAL
[TUNNEL SLA EVENT] s1 <-> s2 CRITICAL -> NORMAL
Go to Settings to activate Windows.
```

نلاحظ أن النظام اتخذ قرار بالإبقاء على المسار الحالي للنفق بين S1 و S2، في حين قرر إعادة توجيه حركة المرور الخاصة بالنفق بين S2 و S4 عبر المسار البديل الذي يمر من خلال S3. ويعتمد هذا القرار على الأوزان التي يستخدمها نموذج التعلم المعزز (RL) لتقييم المسارات البديلة واختيار الإجراء الأنسب لكل حالة. ومع تكرار عمليات اتخاذ القرار، يتم تحديث هذه الأوزان وتحسينها من خلال آلية التعلم المباشر (Online Learning) بعد كل تجربة، مما يسمح للنظام بتحسين جودة قرارات إعادة التوجيه تدريجياً بناءً على نتائج القرارات السابقة.

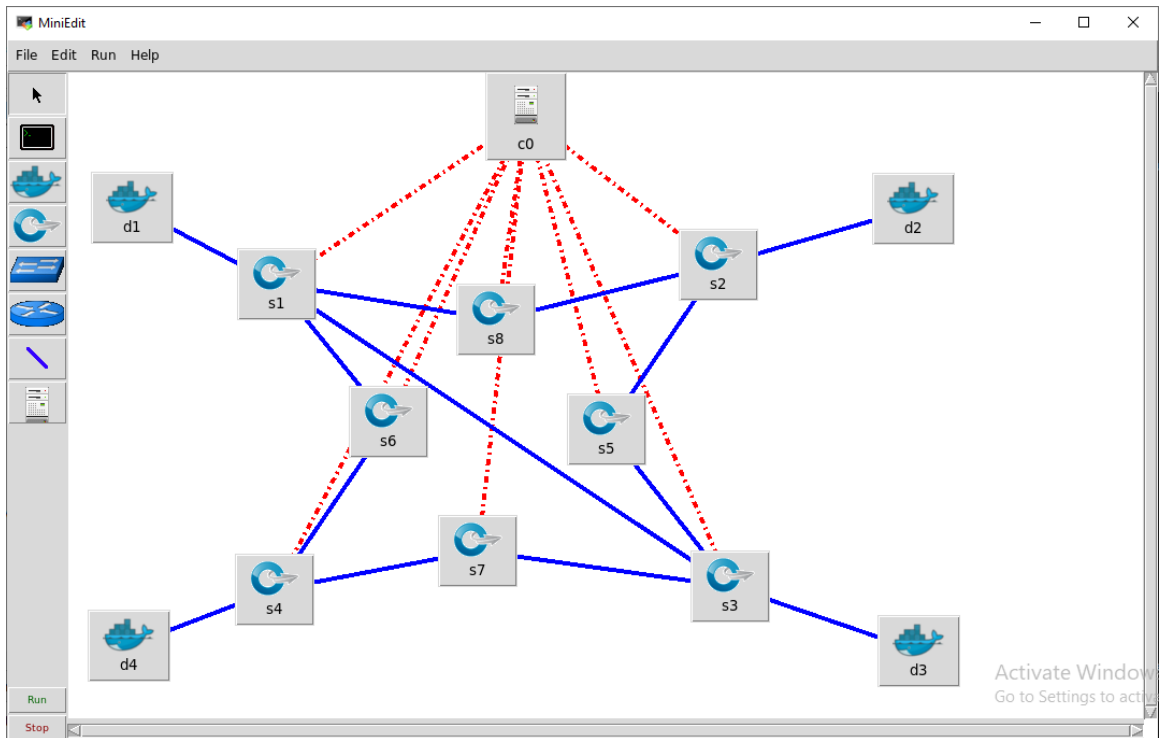
2. الاختبار على الطوبولوجيا الثانية (8 سويتشات):

بتكرار نفس الخطوات السابقة :

تشغيل المتحكم :

```
root@shouib:/home/shouib/SD-WAN# ryu-manager --observe-links controller/topology_aware_switch.py controller/link_metrics_app.py
ryu.app.ofctl_rest ryu.app.rest_topology
loading app controller/topology_aware_switch.py
loading app controller/link_metrics_app.py
loading app ryu.app.ofctl_rest
loading app ryu.app.rest_topology
loading app ryu.topology.switches
loading app ryu.controller.ofp_handler
creating context wsgi
instantiating app None of DPSet
creating context dpset
instantiating app controller/topology_aware_switch.py of TopologyAwareSwitch
instantiating app controller/link_metrics_app.py of LinkMetricsApp
[link_metrics] /metrics exposed on :9200
instantiating app ryu.app.ofctl_rest of RestStatsApi
instantiating app ryu.app.rest_topology of TopologyAPI
instantiating app ryu.topology.switches of Switches
instantiating app ryu.controller.ofp_handler of OFPHandler
(17867) wsgi starting up on http://0.0.0.0:8080
Activate Windows
Go to Settings to activate Windows.
```

اختيار الطوبولوجيا و تشغيلها :



الشكل 9: الطوبولوجيا 2

```

(17867) wsgi starting up on http://0.0.0.0:8080
[topology] 1 switches, 0 directed links
[topology] 2 switches, 0 directed links
[topology] 3 switches, 0 directed links
[topology] 3 switches, 1 directed links
[topology] 4 switches, 1 directed links
[topology] 4 switches, 2 directed links
[topology] 5 switches, 2 directed links
[topology] 5 switches, 4 directed links
[topology] 6 switches, 4 directed links
[topology] 6 switches, 4 directed links
[topology] 6 switches, 7 directed links
[topology] 6 switches, 8 directed links
[topology] 6 switches, 10 directed links
[topology] 7 switches, 10 directed links
[topology] 7 switches, 10 directed links
[topology] 7 switches, 10 directed links
[topology] 7 switches, 12 directed links
[topology] 8 switches, 13 directed links
[topology] 8 switches, 13 directed links
[topology] 8 switches, 14 directed links
[topology] 8 switches, 14 directed links
[topology] 8 switches, 17 directed links
[topology] 8 switches, 17 directed links
[topology] 8 switches, 17 directed links
[topology] 8 switches, 18 directed links

```

اكتشاف المتحكم تلقائيا للطوبولوجيا :

تشغيل سكربت بناء الأنفاق :

```
shouib@shouib:~/SD-WAN$ sudo python3 -m telemetry.tunnel_manager
[sudo] password for shouib:
Using database: /home/shouib/SD-WAN/database/telemetry.db
[tunnel] TunnelManager started
[tunnel] edge switches: [1, 2, 3, 4]
[tunnel] creating VXLAN s1<->s2
[tunnel] CREATED s1<->s2 logical-vni=11002 directional-vni=11002/12001
[tunnel] creating VXLAN s1<->s3
[tunnel] CREATED s1<->s3 logical-vni=11003 directional-vni=11003/13001
[tunnel] creating VXLAN s1<->s4
[tunnel] CREATED s1<->s4 logical-vni=11004 directional-vni=11004/14001
[tunnel] creating VXLAN s2<->s3
[tunnel] CREATED s2<->s3 logical-vni=12003 directional-vni=12003/13002
[tunnel] creating VXLAN s2<->s4
[tunnel] CREATED s2<->s4 logical-vni=12004 directional-vni=12004/14002
[tunnel] creating VXLAN s3<->s4
[tunnel] CREATED s3<->s4 logical-vni=13004 directional-vni=13004/14003
[tunnel] edge switches: [1, 2, 3, 4]
```

شغيل تطبيق المراقبة وجمع القياسات :

```
shouib@shouib:~/SD-WAN$ python3 -m agents.telemetry_agent_runner
Using database: /home/shouib/SD-WAN/database/telemetry.db
[sla_collector] /metrics exposed on :9201
```

نقوم بمحاكاة حالة حقن عطل تأخير (200 ms) على الوصلة 1-8 :

```
shouib@shouib:~/SD-WAN$ python3 chaos/run_scenario.py --link 1 8 --delay 200 --duration 60
Using database: /home/shouib/SD-WAN/database/telemetry.db
[tc] sudo tc qdisc replace dev s1-eth3 root netem delay 200.0ms
[tc] sudo tc qdisc replace dev s8-eth1 root netem delay 200.0ms
[chaos] fault active on s1<->s8 for 60.0s
[tc] sudo tc qdisc del dev s1-eth3 root
[tc] sudo tc qdisc del dev s8-eth1 root
[chaos] fault cleared on s1<->s8
```

اكتشاف النظام للأنفاق المتأثرة و اتخاذ قرار اعادة التوجيه :

```
shouib@shouib:~/SD-WAN$ python3 -m agents.telemetry_agent_runner
Using database: /home/shouib/SD-WAN/database/telemetry.db
[sla_collector] /metrics exposed on :9201
[TUNNEL SLA EVENT] s1 <-> s2 NORMAL -> CRITICAL
[EXECUTE][REROUTE] OK: Rerouted tunnel s1<->s2 onto ['s1', 's3', 's5', 's2'] (priority=100, idle_timeout=300s), forward on ['s1', 's3', 's5'], reverse on ['s2', 's5', 's3']
[GRAPH] tunnel state: CRITICAL action: REROUTE approved: True executed: True detail: Rerouted tunnel s1<->s2 onto ['s1', 's3', 's5', 's2'] (priority=100, idle_timeout=300s), forward on ['s1', 's3', 's5'], reverse on ['s2', 's5', 's3']
[TUNNEL SLA EVENT] s1 <-> s2 CRITICAL -> NORMAL
```

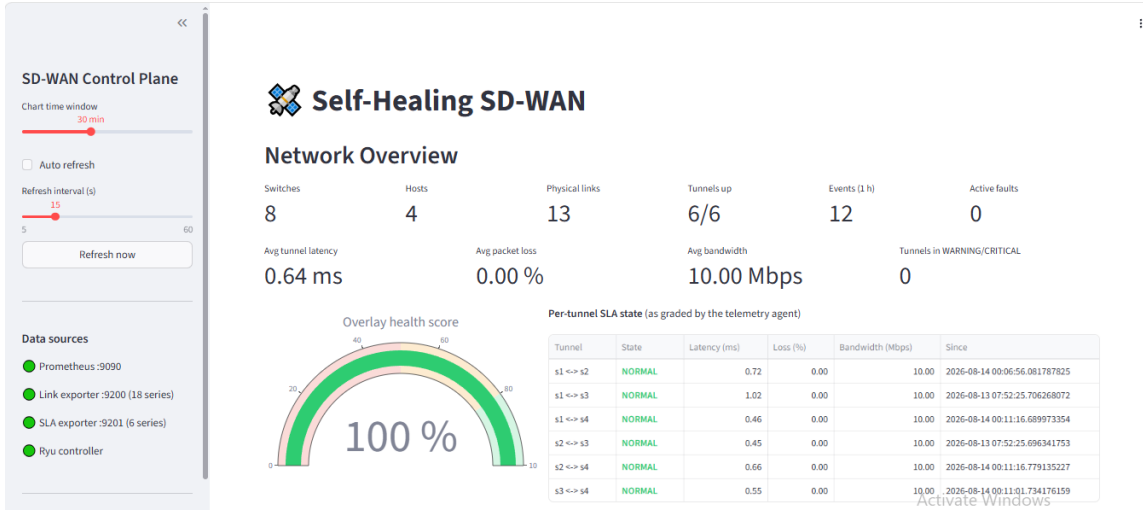
3. اختبار ال dashboard :

```
shouib@shouib:~/SD-WAN$ streamlit run dashboard/app.py
2026-08-13 17:11:54.800 Uvicorn server started on 127.0.0.1:8501

You can now view your Streamlit app in your browser.

URL: http://127.0.0.1:8501
```

نلاحظ أن ال dashboard تظهر لنا لمحة عامة عن الشبكة و حالة الأنفاق :

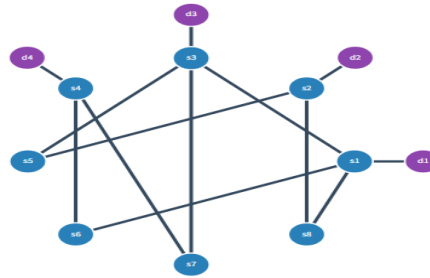


الشكل 10: لمحة عامة عن الشبكة

بالإضافة لاتاحة عرض الطوبولوجيا سواء شبكة overlay أو underlay :

Live Topology

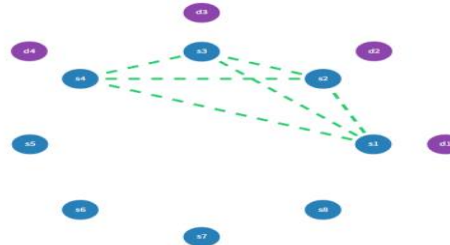
☒ Underlay (physical links) ☐ Overlay (VXLAN tunnels)



الشكل 12: underlay topology

Live Topology

☐ Underlay (physical links) ☒ Overlay (VXLAN tunnels)



الشكل 11: overlay topology

أيضا يوجد عرض لمؤشرات الأنفاق اخر 30 دقيقة من prometheos :



شكل 13: مؤشرات أداء الأنفاق

هـ أيضا عرض لمؤشرات الأداء الخاصة بالوصلات الفيزيائية :



شكل 14: مؤشرات أداء الوصلات الفيزيائية

بالإضافة لتوفير لوحة تحكم خاصة لمحاكاة حقن الأعطال :

Fault Injection (tc netem)

Shells out to `sudo tc / sudo ip` on the interfaces backing the selected link. Impairment is applied to BOTH ends, so it affects traffic in either direction.

Target link

s1 <-> s3

Delay (ms) 0 Jitter (ms) 0 Loss (%) 0.00 Rate limit (Mbit/s) 0.00

Auto-clear after (s) — 0 keeps the fault until cleared by hand

Inject fault Clear this link Link DOWN Link UP

Clear ALL faults on every link

الشكل 15: لوحة التحكم الخاصة بمحاكاة حقن الأعطال

و سجل يظهر اعمليات محاكاة الأعطال التي تم تطبيقها :

Fault history

Link	Type	Impairment	Started	Ended	Duration (s)	Active
s1 <-> s3	netem	delay 200.0ms	2026-08-14 00:05:54	2026-08-14 00:06:54	60.4	☐

الشكل 16: سجل عمليات المحاكاة

و سجل يظهر القرارات المتخذة من النظام :

Decision log

Time	Metric	State	Action	Reasoning	Approved	Guardrail	Executed	Result
2026-08-14 00:06:26	tunnel	CRITICAL	REROUTE	RL scorer picked REROUTE (score=0.221, exploitation)	yes	Selected path verified among candidates	yes	Rerouted tunnel
2026-08-13 23:58:48	tunnel	CRITICAL	NONE	RL scorer chose to stay on the active path (score 0.085 beat every alternative)	no	No action was proposed	no	None
2026-08-13 23:58:46	tunnel	CRITICAL	NONE	RL scorer chose to stay on the active path (score 0.028 beat every alternative)	no	No action was proposed	no	None
2026-08-13 23:58:45	tunnel	CRITICAL	NONE	RL scorer chose to stay on the active path (score 0.060 beat every alternative)	no	No action was proposed	no	None
2026-08-13 23:27:09	tunnel	CRITICAL	REROUTE	RL scorer picked REROUTE (score=0.223, exploitation)	yes	Selected path verified among candidates	yes	Rerouted tunnel
2026-08-13 23:27:08	tunnel	CRITICAL	NONE	RL scorer chose to stay on the active path (score 0.227 beat every alternative)	no	No action was proposed	no	None
2026-08-11 12:47:42	tunnel	CRITICAL	NONE	No alternative path exists to reroute onto (no candidate paths, unsupported metric_type, no		No action was proposed	no	None
2026-08-11 12:47:42	tunnel	CRITICAL	NONE	No alternative path exists to reroute onto (no candidate paths, unsupported metric_type, no		No action was proposed	no	None

Export decisions as CSV

Activate Windows

الشكل 17: سجل القرارات المتخذة

و سجل يظهر الأحداث التي تم إطلاقها و اكتشافها :

Event Log

Events shown

100

Severity

CRITICAL x INFO x WARNING x

Time	Type	Switch	Port	Severity	Description
2026-08-14 00:11:16.77953189	TUNNEL_RECOVERY		2	4 INFO	tunnel s2 <-> s4: Tunnel SLA back to normal: latency=0.581 ms, jitter=0.873 ms, loss=0.0%, bandwidth=100.0%
2026-08-14 00:11:16.69069577	TUNNEL_RECOVERY		1	4 INFO	tunnel s1 <-> s4: Tunnel SLA back to normal: latency=4.025 ms, jitter=7.647 ms, loss=0.0%, bandwidth=100.0%
2026-08-14 00:11:01.734822989	TUNNEL_RECOVERY		3	4 INFO	tunnel s3 <-> s4: Tunnel SLA back to normal: latency=0.377 ms, jitter=0.415 ms, loss=0.0%, bandwidth=100.0%
2026-08-14 00:06:56.082333803	TUNNEL_RECOVERY		1	2 INFO	tunnel s1 <-> s2: Tunnel SLA back to normal: latency=0.246 ms, jitter=0.073 ms, loss=0.0%, bandwidth=100.0%
2026-08-14 00:06:25.537554979	TUNNEL_SLA_VIOLATION		1	2 CRITICAL	tunnel s1 <-> s2: Tunnel SLA violated: latency=401.464 ms, jitter=0.562 ms, loss=0.0%, bandwidth=100.0%
2026-08-13 23:58:46.631552219	TUNNEL_SLA_VIOLATION		2	4 CRITICAL	tunnel s2 <-> s4: Tunnel SLA violated: latency=None ms, jitter=None ms, loss=100.0%, bandwidth=0.0%
2026-08-13 23:58:45.616927147	TUNNEL_SLA_VIOLATION		3	4 CRITICAL	tunnel s3 <-> s4: Tunnel SLA violated: latency=None ms, jitter=None ms, loss=100.0%, bandwidth=0.0%
2026-08-13 23:58:44.261300325	TUNNEL_SLA_VIOLATION		1	4 CRITICAL	tunnel s1 <-> s4: Tunnel SLA violated: latency=None ms, jitter=None ms, loss=100.0%, bandwidth=0.0%
2026-08-13 23:28:08.155643940	TUNNEL_RECOVERY		1	2 INFO	tunnel s1 <-> s2: Tunnel SLA back to normal: latency=0.984 ms, jitter=1.625 ms, loss=0.0%, bandwidth=100.0%
2026-08-13 23:27:37.185902596	TUNNEL_RECOVERY		2	4 INFO	tunnel s2 <-> s4: Tunnel SLA back to normal: latency=0.171 ms, jitter=0.021 ms, loss=0.0%, bandwidth=100.0%
2026-08-13 23:27:08.344121695	TUNNEL_SLA_VIOLATION		2	4 CRITICAL	tunnel s2 <-> s4: Tunnel SLA violated: latency=0.563 ms, jitter=0.398 ms, loss=60.0%, bandwidth=10.0%

Export events as CSV

Activate Windows
Go to Settings to activate Windows.

الشكل 18: سجل أحداث الشبكة

3.4- خلاصة وآفاق مستقبلية

يمكن تلخيص الآفاق المستقبلية للمشروع في النقاط التالية:

- تطوير النموذج الذكي: استبدال النموذج الخطي بشبكة عصبية عميقة (Deep Q-Network) لالتقاط علاقات أكثر تعقيداً بين مقاييس الشبكة .
- إثراء دالة المكافأة: إدراج معايير إضافية كتكلفة الرابط، عدد القفزات، وتوازن الحمل، وليس الاكتفاء بمؤشرات SLA الزمنية فقط.
- التعلم المعزز متعدد العملاء: الانتقال من عميل قرار مركزي إلى عدة عملاء متعاونين للشبكات واسعة النطاق .
- اختبار قابلية التوسع: تقييم أداء النظام على شبكات أكبر تضم عشرات المفاتيح .
- تعزيز الأمان: إضافة مصادقة وتشفير TLS لواجهات REST ولوحة التحكم، بدل الاقتصار على الوصول المحلي .
- الانتشار في بيئة سحابية حقيقية: اختبار النظام خارج المحاكاة، على بنية تحتية متعددة المناطق بروابط WAN فعلية .
- اختبار الحمل الإنتاجي: قياس سلوك النظام تحت حركة بيانات واقعية بدلاً من فحوصات محدودة (ping/iperf3) .